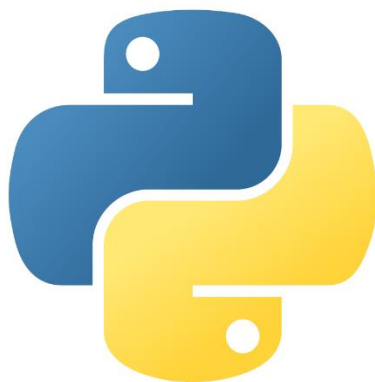
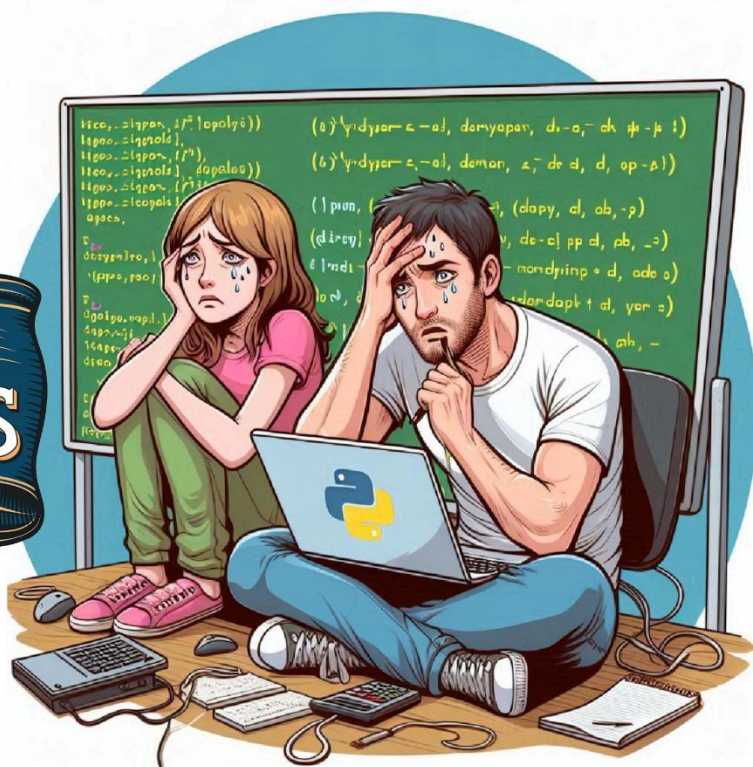


Aprende **UN POCO** de **PYTHON** haciendo ejercicios



¡Con esfuerzo!
¡Con trabajo!
¡Con dedicación!



Ejercicios disponibles en GitHub

Incluye apoyo moral en



Fernando Paniagua

Aprende UN POCO de Python

haciendo ejercicios



© 2025 by Fernando Paniagua
fernando.paniagua.python@gmail.com

Contenido

Introducción	6
Ejercicio 1 – La función print	8
Ejercicio 2 – Función input	9
Ejercicio 3 – Entorno interactivo	10
Ejercicio 4 – ¿Qué es eso de f-string ?	11
Ejercicio 5 – Crear y recorrer una lista	12
Ejercicio 6 – Crear una tupla y convertirla a lista	14
Ejercicio 7 – Manejando conjuntos	15
Ejercicio 8 – Diccionario de sinónimos	16
Ejercicio 9 – Función de suma	17
Ejercicio 10 – Función con argumentos con valores por defecto	19
Ejercicio 11 – Evaluación de clientes	20
Ejercicio 12 – Ordenación de una lista.....	22
Ejercicio 13 – Obtención de valores numéricos y ordenación.	23
Ejercicio 14 – Conversión de tipos básicos	25
Ejercicio 15 – Comprobación de elementos de una lista	26
Ejercicio 16 – Obtención los valores mayores y menores de una lista.....	28
Ejercicio 17 – Paso de parámetros por referencia	29
Ejercicio 18 – Paso de parámetros por valor.....	30
Ejercicio 19 – Generación de listas mediante list comprehension	31
Ejercicio 20 – Expresión ternaria	33
Ejercicio 21 – Obtención de una lista con los elementos de una tupla que cumplen una determinada condición.	34
Ejercicio 22 – Ordenar los elementos de una tupla (ERROR)	35
Ejercicio 23 – Ordenar los elementos de una tupla (CORRECTO).....	36

Ejercicio 24 – Funciones que reciben muchos argumentos.....	37
Ejercicio 25 – Adivina, adivinanza	39
Ejercicio 26 – Escritura de datos en un fichero	41
Ejercicio 27 – Lectura de datos de un fichero	42
Ejercicio 28 – Creación de un fichero de configuración utilizando JSON	43
Ejercicio 29 – Lectura de un fichero de configuración en formato JSON	45
Ejercicio 30 – Calculadora.....	47
Ejercicio 31 – El módulo math	49
Ejercicio 32 – En busca del par perdido: la sentencias break y else	50
Ejercicio 33 – Paso de todo. La sentencia pass	52
Ejercicio 34 – Destruir al monstruo con la sentencia match	53
Ejercicio 35 – Vayamos por partes: el método split	55
Ejercicio 36 – Lectura de ficheros csv	56
Ejercicio 37 – Aplicación gráfica con tkinter	58
Ejercicio 38 – Programación funcional: la función filter	60
Ejercicio 39 – Programación funcional: la función map	61
Ejercicio 40 – Programación funcional: la función reduce	62
Ejercicio 41 – Programación orientada a objetos: definición de clase.....	64
Ejercicio 42 – Programación orientada a objetos: instanciación de objetos e invocación de métodos	66
Ejercicio 43 – Métodos mágicos __str__ y __repr__	68
Ejercicio 44 – Programación orientada a objetos: list-comprehension	70
Ejercicio 45 – Programación orientada a objetos: función filter	72
Ejercicio 46 – Programación orientada a objetos: función map	74
Ejercicio 47 – Programación orientada a objetos: función reduce	76
Ejercicio 48 – Funciones lambda	78

Ejercicio 49 – Persistencia de objetos con el módulo pickle	80
Ejercicio 50 – Introducción a las bases de datos con SQLite	82
Ejercicio 51 – ¿Tienes hora?	85
Ejercicio 52 – ¿Cuántos años tengo?	86
Ejercicio 53 – Propósito	88
Ejercicio 54 – Manteniendo el tipo	89
Ejercicio 55 – Excepciones	91
Ejercicio 56 – Comprobando tipos	93
Ejercicio 57 – Aserciones. ¿Eso qué es?	95
Ejercicio 58 – Almacenamiento de funciones en variables	96
Ejercicio 59 – Estructuras de datos: creando una cola	98
Ejercicio 60 – Búsqueda de palabras	100
Ejercicio 61 – Parando el tiempo	102
Ejercicio 62 – Organización de proyectos: módulos	103
Ejercicio 63 – *args	105
Ejercicio 64 – **kwargs	106
Ejercicio 65 – Formateando cadenas con el método format	107
Ejercicio 66 – Trabajando en Unix/Linux/macOS: shebang	109
Ejercicio 67 – Procesando textos: El Quijote	110
Ejercicio 68 – Obteniendo datos de la web con request	113
CONCLUSIONES	115

Introducción

Habiendo impartido unos cuantos (muchos) cursos de programación en **Python**, me entraron ganas de escribir un libro para que aquellas personas que quieran comenzar en el bonito y sufrido mundo de la programación en este lenguaje lo hagan de una forma “amigable”.

En la portada aparecen unos eslóganes muy poco estimulantes. Hacen referencia al esfuerzo, a la dedicación y a la ausencia de milagros. ¿Cómo que se puede aprender a programar en **Python** en un fin de semana? ¿Quién se cree que sin esfuerzo se logran grandes objetivos? ¡Basta de patrañas! ¡Ya está bien! Aprender requiere sacrificio y tiempo. **Python** es un lenguaje de programación algo más “sencillo” que otros lenguajes, pero eso no implica que podamos llegar de novato a maestro en quince días. Al menos yo no lo creo. Por eso he decidido ir de frente y no hacer falsas promesas: después de leer y trabajar con este libro sabrás **un poco de Python**, lo suficiente para tener claro si te gusta o no la programación en general y este lenguaje en particular. Te habrás metido en el agua hasta la cintura y tendrás algo más de criterio para decidir si te tiras de cabeza o te das la vuelta y te vas al chiringuito más cercano.

Como ya hay muchos libros, he dedicado un rato (poco) a pensar en cómo hacer algo distinto. En base a esa breve reflexión he concluido que:

- No voy a indicar como **instalar Python**. Lo enseñaré en un vídeo en **YouTube**.
- No voy a mostrar como **instalar Visual Studio Code**, el editor que recomiendo para escribir y probar los ejemplos. Haré lo mismo que con la instalación: **YouTube**.
- No voy a comenzar explicando qué es **Python**, sus características y bondades (y maldades). En este asunto particular delego en Wikipedia y/o en ChatGPT o similares.

La metodología de trabajo va a ser la siguiente: propongo un ejercicio, lo resuelvo y lo explico. Así suelo trabajar en las clases que imparto y los

alumnos aprenden. Cuando explico teoría hacen poco caso, por lo que he decidido aplicar el método “aprender haciendo” que parece que funciona mejor que “aprender mirando el móvil mientras un señor habla de cosas que no me interesan”.

Otra regla del libro es que cuando llegue a las 100 páginas (más o menos) estará terminado. Me dan repelús los libros esos de 850 páginas, así que voy a jugar en corto. Si el formato, tanto en el fondo como la forma, tiene éxito, habrá sido el comienzo de la colección “SIN MILAGROS” y publicaré libros similares dedicados a otros lenguajes de programación y tecnologías relacionadas con el desarrollo de software. Si no es así, nos vemos en el chiringuito.

Empecé a escribir este libro un día 13 de febrero de 2025 en Madrid. El día estaba fresco pero soleado y tenía instalada la versión 3.13 de Python. Lo terminé el día 6 de abril de 2025 en Alcorcón, Madrid. También era un día soleado aunque algo más primaveral.

Los vídeos de apoyo/explicación los puedes encontrar en:

- <https://www.youtube.com/@fernandopaniaguaformacion>

El repositorio de GitHub con los ejercicios se encuentra en:

- <https://github.com/fpaniaguaformacion/aprende-un-poco-de-python-ejercicios>

```
1 print( 'Comencemos' )
```

Ejercicio 1 – La función **print**

Enunciado:

Mostrar por pantalla el texto **Hola mundo**.

Solución:

```
1 # Mostrar por consola el texto 'Hola mundo'
2
3 print('Hola mundo')
4
```

Explicación:

En la línea 1 aparece un texto precedido por una almohadilla (#). La almohadilla indica que el texto que se encuentra a continuación no forma parte del programa, solo es un texto informativo. No hace nada, vaya. Esto se conoce como línea de comentario.

La línea 2 está vacía. A partir de ahora las líneas vacías no las comentaremos ya que son 'auto explicativas'. Como no tienen nada, no hacen nada. Es lo esperable.

En la línea 3 aparece una llamada a la función **print**. Esta es una de las funciones integradas o *built-in* de Python. La lista completa está en esta URL: <https://docs.python.org/3/library/functions.html>

La función **print**, como muchas otras, admite varias formas de ser llamada. En este caso se muestra la más simple de todas, que es pasando una cadena de caracteres como argumento. Esta cadena de caracteres puede estar delimitada por comillas simples o dobles, da igual. Ejecuta el ejercicio y comprueba que la salida es correcta.

Ejercicio 2 – Función **input**

Enunciado:

Solicitar al usuario un texto y almacenarlo en una variable.

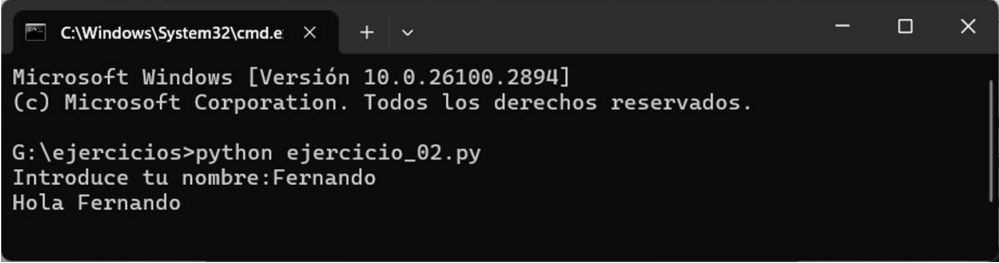
Solución:

```
1 nombre = input('Introduce tu nombre:')
2 print('Hola', nombre)
3
```

Explicación:

En la línea 1 se utiliza la función **input** que muestra por pantalla el texto que se pasa como argumento y recoge en la variable **nombre** el texto introducido por el usuario.

La línea 2 se hace una llamada a la función **print** pasándole dos argumentos separados por coma: el texto 'Hola' y la variable **nombre**. Esto produce que se muestre por pantalla el texto y el valor de la variable separados por un espacio en blanco (la coma es la responsable).



```
C:\Windows\System32\cmd.e  X  +  v  -  □  X
Microsoft Windows [Versión 10.0.26100.2894]
(c) Microsoft Corporation. Todos los derechos reservados.

G:\ejercicios>python ejercicio_02.py
Introduce tu nombre:Fernando
Hola Fernando
```

Ejercicio 3 – Entorno interactivo

Enunciado:

Acceder al entorno interactivo de Python y mostrar el tipo de una variable.

Solución:

Abrir una terminal de Windows (pulsar el botón de inicio y escribir **Terminal** en el buscador de aplicaciones). Una vez abierta escribir **Python** y pulsar la tecla **Enter**. Una vez en entorno interactivo se pueden escribir sentencias en Python que se ejecutarán cuando se pulse **Enter**.

```
1 Python 3.13.1 (tags/v3.13.1:0671451, Dec 3 2024, 19:06:28) [MSC v.1942 64 bit (AMD64)] on win32
2 Type "help", "copyright", "credits" or "license" for more information.
3 >>> nombre = 'Fernando'
4 >>> type(nombre)
5 <class 'str'>
6 >>>
```

Explicación:

En el entorno interactivo se pueden escribir sentencias y ejecutarlas directamente. Lo bueno es que no hace falta crear ficheros. Lo malo es que lo escrito se pierde cuando se cierre el entorno, por lo que tendremos que utilizarlo para hacer pruebas y para aprender.

En las líneas 3 y 4 se puede ver que aparece la secuencia de caracteres `>>>`. Es ahí donde hay que escribir las sentencias.

En la línea 3 se declara la variable **nombre** y se le asigna (el símbolo `=` asigna el resultado de la expresión del lado derecho a la variable del lado izquierdo).

En la línea 4 se ejecuta la función **type** y se le pasa como argumento la variable **nombre**. Esta función muestra el tipo de la variable. En este caso es del tipo **str**, como se muestra en la línea 5. **str** es el tipo que almacena cadenas de caracteres.

Ejercicio 4 – ¿Qué es eso de *f-string*?

Enunciado:

Solicitar la introducción de un texto y un número mediante el teclado y mostrar una cadena de caracteres utilizando los valores obtenidos.

Solución:

```
1 nombre = input('Introduce tu nombre:')
2 edad = input('Introduce tu edad:')
3
4 print(f'Hola {nombre}, tienes {edad} años')
5
```

Explicación:

En las líneas 1 y 2 se pide al usuario introducir valores para las variables **nombre** y **edad**. Ojo, porque el tipo de la variable **edad** será **str**, aunque se introduzca un número. Para que sea de tipo numérico hay que hacer una conversión de tipo (ya haremos más adelante algún ejercicio).

La línea 4 es la interesante de este ejercicio. A la función **print** se le pasa como argumento la letra **f** y después una cadena. En dicha cadena, se incluyen los nombres de las variables **nombre** y **edad** rodeados de llaves. El efecto de este galimatías es el siguiente: se genera una cadena de caracteres sustituyendo las **interpolaciones** (lo de las llaves se llama así) por el valor de las variables. A esta forma de crear cadenas (utilizando la letra **f**) se la conoce como **f-string**.

Por ejemplo, si introducimos el nombre **María** y la edad **23** la salida generada por este programa será la siguiente: **Hola María, tienes 23 años**

Ejercicio 5 – Crear y recorrer una lista

Enunciado:

Creación de una lista con los ingredientes de una tortilla de patatas (con cebolla, por supuesto).

Recorrerla y mostrar sus ingredientes por la pantalla.

Solución:

```
1 ingredientes_tortilla = ['Huevos', 'Patatas', 'Aceite', 'Sal']
2 ingredientes_tortilla.append('Cebolla')
3
4 for ingrediente in ingredientes_tortilla:
5     print(ingrediente)
```

Explicación:

Una lista es una colección de elementos heterogéneos (pueden ser de diferentes tipos). Se puede crear de distintas maneras. Una es mediante el uso de corchetes. Los elementos deben ir separados por comas.

En la línea 1 se declara una lista con el nombre **ingredientes_tortilla** inicializada con unos cuantos elementos. Los nombres de las variables se deben escribir en letras minúsculas y si contienen varias palabras estas irán separadas por un guion bajo.

En la línea 2 se añade al final de la lista un nuevo elemento (la cebolla) a la lista mediante el uso del método **append**.

La línea 4 contiene un bucle **for** que recorre los elementos de la lista **ingredientes_tortilla**. La sintaxis del bucle **for** es la siguiente:

```
1 for nombre_elemento in nombre_lista:
```

Aprende un poco de Python

La línea 5 está sangrada 4 caracteres hacia la derecha, lo que quiere decir que se va a ejecutar tantas veces como elementos tenga la lista. En este caso, se muestra por pantalla el valor de la variable **ingrediente**, que es como se ha indicado en la línea 4 que se llama la variable que recoge cada uno de los elementos de la lista según la iteración en la que se encuentre.

El resultado de la ejecución es el siguiente:

A screenshot of a terminal window with a dark background. It displays a list of five ingredients, each preceded by a line number from 1 to 5. The text is as follows:

```
1 Huevos
2 Patatas
3 Aceite
4 Sal
5 Cebolla
```

Se puede observar que el elemento 'Cebolla' se encuentra en el último lugar, ya que fue el añadido con el método **append**.

También se puede comprobar que se ha ejecutado 5 veces la línea 5, y que la función **print** inserta un salto de línea después de mostrar el contenido de la variable.

Ejercicio 6 – Crear una tupla y convertirla a lista

Enunciado:

Pues eso, hay que crear una tupla y convertirla en una lista para poder añadir un nuevo elemento.

Solución:

```
1 dias = ('Luns', 'Martes', 'Mércores', 'Xoves')
2 dias = list(dias)
3 dias.append('Venres')
```

Explicación:

Las tuplas, al igual que las listas, permiten almacenar en memoria un conjunto de elementos de distintos tipos. Pero al contrario de las listas, no se pueden modificar porque son INMUTABLES.

En la línea 1 se está creando una tupla con los días de diario en gallego. Se puede observar que se utilizan paréntesis para delimitar los elementos de la tupla que van separados por comas (si una tupla solo contiene un elemento también llevará coma al final o el resultado no será una tupla). Pero se nos ha olvidado incluir el viernes (venres, en gallego) y la tupla no admite cambios. ¿Qué hacer?

La solución está en la línea 2. La función **list** permite convertir determinadas estructuras de datos como la tupla en una lista. En esta línea se está asignando la lista obtenida a partir de la tupla **dias** a una variable con el mismo nombre.

Ahora sí, en la línea 3 se puede invocar al método **append** y agregar el día de la semana que se nos había extraviado.

Ejercicio 7 – Manejando conjuntos

Enunciado:

Estoy en una tienda en la que venden videoconsolas. Ya tengo algunas, pero quiero obtener el conjunto de las consolas que no tengo para saber qué me puedo comprar.

Solución:

```
1 consolas_disponibles = {'PlayStation 5', 'XBOX Series', 'Nintendo Switch 2', 'Steam Deck'}
2 mis_consolas = {'XBOX Series', 'Steam Deck', 'Steam Deck'}
3
4 print(mis_consolas) # {'Steam Deck', 'XBOX Series'}
5
6 consolas_candidatas = consolas_disponibles.difference(mis_consolas)
7
8 print(consolas_candidatas) # {'Nintendo Switch 2', 'PlayStation 5'}
```

Explicación:

Un conjunto es una estructura de datos de cualquier tipo que no admite duplicados y en la que el orden no importa. Se delimita utilizando llaves y los elementos van separados por comas, como en las listas y las tuplas.

En las líneas 1 y 2 se declaran e inicializan dos conjuntos. El conjunto **mis_consolas** tiene dos '**Steam Deck**' (yo solo tengo una), pero como se puede ver en el comentario que hay en la línea 4, correspondiente con el resultado de ejecutar dicha línea, sólo aparece una vez. Recuerda: los conjuntos no admiten duplicados. También fíjate en que el orden no es el mismo de la inicialización. Esto es porque en los conjuntos el orden no es relevante.

Las operaciones que se pueden acceder con las estructuras de datos **set** es la propia de los conjuntos, como por ejemplo obtener la relación de elementos de un conjunto que no están en otros (la diferencia). En la línea 6 se puede ver cómo obtener el conjunto de elementos de **consolas_disponibles** que no está en **mis_consolas** utilizando el método **difference**. En el comentario de la línea 8 se muestra el resultado.

Ejercicio 8 – Diccionario de sinónimos

Enunciado:

Crear un diccionario de sinónimos.

Solución:

```
1 sinonimos = {'Feliz': ('Contento', 'Alegre', 'Jubiloso'),  
2             'Triste': ('Infeliz', 'Desgraciado', 'Melancólico'),  
3             'Inteligente': ('Astuto', 'Perspicaz', 'Sagaz')}  
4  
5 print(sinonimos['Triste']) # ('Infeliz', 'Desgraciado', 'Melancólico')  
6
```

Explicación:

Un diccionario, también conocido como array asociativo, mapa o *hash table*, es una estructura de datos que relaciona claves con valores.

En Python, el diccionario (tipo **dict**) se delimita por llaves y los elementos se separan por comas. Cada elemento se compone de una clave y un valor separados por dos puntos (:).

En este ejercicio, se define un diccionario de sinónimos en el que las claves son **Feliz**, **Triste** e **Inteligente**. Los valores asociados a cada una de dichas claves son los sinónimos, agrupados en una tupla.

En la línea 5 se puede observar como se accede al valor de una clave concreta. En este caso, a los sinónimos de **Triste**.

Ejercicio 9 – Función de suma

Enunciado:

Crear una función que sume dos números.

Solución:

```
1 def sumar(sumando1, sumando2):  
2     resultado = sumando1 + sumando2  
3     return resultado  
4  
5 resultado = sumar(10, 19)  
6 print(resultado) # 29  
7
```

Explicación:

Una función es, en programación, una agrupación de código identificado con un nombre que puede utilizarse una y otra vez. Puede recibir argumentos para que pueda adaptarse a diferentes escenarios y devolver un resultado si es oportuno.

En Python, las funciones se identifican con la palabra **def** seguida del nombre de la función, los argumentos entre paréntesis y dos puntos. A continuación, sangrado hacia la derecha, el bloque de código que forma el cuerpo de la función. Los argumentos son opcionales, pero siempre tienen que estar indicados los paréntesis y los dos puntos, ya que forman parte de la sintaxis de la función.

Dentro del bloque de la función, se realiza la operación para la que fue diseñada y, opcionalmente, se devuelve un resultado mediante la instrucción **return**.

En este ejemplo, la función tiene como nombre **sumar** y recibe los argumentos **sumando1** y **sumando2**. En el cuerpo de la función se suman dichos argumentos y se almacena el resultado en la variable **resultado**, que es devuelta en la línea 3 con la instrucción **return**.

En la línea 5 se hace una llamada a la función, indicando los valores del argumento y recogiendo el resultado en una variable cuyo valor se muestra en la línea 6.

Ejercicio 10 – Función con argumentos con valores por defecto

Enunciado:

Crear una función que recibe como argumento un nombre y muestra un mensaje por pantalla. Este argumento es opcional (tiene un valor por defecto).

Solución:

```
1 def saludar(nombre = 'Desconocido'):  
2     print(f'Hola {nombre}')  
3  
4 saludar('Rosa') # Hola Rosa  
5 saludar() # Hola Desconocido  
6
```

Explicación:

En este caso, la función recibe el argumento **nombre** al que se le asigna un valor por defecto mediante una indicación en la propia definición de los argumentos (línea 1).

Esto implica que a la función se le puede llamar pasándole un valor al argumento (línea 4) o no (línea 5) en cuyo caso se tomará el valor por defecto indicado en la definición de dicha función.

En la línea 2 se hace uso de nuevo del mecanismo para dar formato a las cadenas de caracteres conocido como *f-string*. Pero eso ya lo habíamos comentado anteriormente ¿verdad?

Ejercicio 11 – Evaluación de clientes

Enunciado:

En una sala de fiestas sólo pueden entrar los mayores de 18 años. Escribir un programa que determine si la edad de una persona es apta para entrar a la sala.

Solución:

```
1 edad = int(input('Introduce la edad del cliente:'))
2 if edad >= 18:
3     print('El cliente es mayor de edad')
4 else:
5     print('La persona no pudo acceder al local')
6
```

Salida:

```
Introduce la edad del cliente:23
El cliente es mayor de edad
```

Explicación:

En la línea 1 se solicita al usuario la introducción de un valor que se corresponderá con la edad del cliente mediante la función **input**. Esta función SIEMPRE devuelve una cadena de caracteres (tipo **str**), por lo que si se quiere trabajar con el dato obtenido como si fuese un número entero, se debe convertir mediante la función **int**. Por lo tanto, **int**, al “envolver” el resultado obtenido de la ejecución de **input**, convierte la cadena introducida por el cliente a un tipo entero o **int** (si no se puede convertir producirá un error de ejecución).

En la línea 2, se evalúa si el valor de la variable para comprobar si es mayor o igual que 18 (operador lógico **>=**) mediante la palabra clave **if**. Como en

otras ocasiones, los dos puntos del final de la línea indican que a continuación se encuentra el bloque de código que se ejecutará si la condición se cumple.

En la línea 4 aparece, en la misma columna que la sentencia **if**, la palabra **else**, junto con el delimitador de bloque. Dicho bloque se ejecutará sí y solo sí la condición de la sentencia **if** no se cumple.

Hay que saber que el bloque **else** es opcional, al igual que la cebolla en la tortilla de patatas.

Ejercicio 12 – Ordenación de una lista

Enunciado:

Ordenar una lista de números.

Solución:

```
1 importes = [100, 200, 30, 25, -10, 800]
2 importes.sort()
3 print(importes) # [-10, 25, 30, 100, 200, 800]
4
```

Explicación:

El método **sort** de clase **list**, ordena los elementos de una lista, modificándola. Existe una función incorporada (*built-in*) que se llama **sorted** que también ordena una lista, pero en este caso generando una nueva con el resultado de dicha ordenación.

En la línea 2 se puede observar cómo se invoca al método **sort** de la lista **importes**. El resultado se muestra en el comentario de la línea 3.

Ojo, porque el método **sort** ordena bien las listas de números, pero si contienen otro tipo de elementos puede no saber cómo ordenar o, sencillamente, provocar un error. Si quieres probar, cambia el elemento numérico 200 por la cadena "200" y prueba a ejecutar el programa.

Ejercicio 13 – Obtención de valores numéricos y ordenación.

Enunciado:

Pedir al usuario un número de valores numéricos, añadirlos a una lista y ordenarlos.

Solución:

```
1 NUMERO_ELEMENTOS = 5
2 numeros = list()
3 for i in range(NUMERO_ELEMENTOS):
4     numero = int(input(f'Introduce el número {i+1}: '))
5     numeros.append(numero)
6 numeros_ordenados = sorted(numeros)
7 print(numeros)
8 print(numeros_ordenados)
9
```

Salida:

```
Introduce el número 1: -8
Introduce el número 2: 101
Introduce el número 3: 32
Introduce el número 4: 1971
Introduce el número 5: 3
[-8, 101, 32, 1971, 3]
[-8, 3, 32, 101, 1971]
```

Explicación:

En este ejercicio aparecen nuevos elementos del lenguaje Python. Veamos qué aportan.

En la línea 1 se está declarando una variable cuyo nombre está escrito enteramente en mayúsculas y que utiliza un guion bajo como separador de las palabras. En otros lenguajes esto sería una **constante**, que es algo parecido a una variable, pero cuyo valor no se puede cambiar (por eso no se llama variable, claro). En Python no existen las constantes, pero sí podemos indicar mediante esta notación que queremos que ese elemento del programa sea tratado como tal. El resto de los programadores y de las programadoras se portarán bien y no modificarán el valor. Y si lo hacen serán responsables de las consecuencias.

En la línea 2 se construye una lista mediante llamada a la ¿función? **list**. Más que una función es un constructor, pero se encuentra dentro de la relación de funciones *built-in*. Sea como sea, se está construyendo una lista vacía.

La línea 3 contiene la declaración de un bucle **for** que itera por una estructura conocida como **range** (un rango de números que va entre 0 y `NUMERO_ELEMENTOS - 1`). En este ejercicio, la variable **i** toma los valores 0, 1, 2, 3 y 4, dependiendo de la iteración en la que se encuentre.

La función **input** de la línea 4 solicita al usuario la introducción de un valor que debe ser convertido a entero mediante la función **int**. Este número es añadido a la lista **números** mediante el uso del método **append**, que inserta el valor al final de dicha lista.

En la línea 6 se hace uso de la función **sorted**. Esta función genera una nueva lista ordenada a partir de la lista que se le pasa como argumento.

Ejercicio 14 – Conversión de tipos básicos

Enunciado:

Convertir un valor numérico a cadena de caracteres

Solución:

```
1 numero = 1_500_000
2 print(type(numero)) # <class 'int'>
3 numero = str(numero)
4 print(type(numero)) # <class 'str'>
5
```

Explicación:

Python utiliza una estrategia de asignación de tipos conocida como **tipado dinámico**. Esto significa que el tipo de una variable no se indica de forma explícita, sino que se asigna según el valor que almacene.

En la línea 1 se está asignando un entero a la variable **numero**. Por lo tanto, el tipo de la variable es **int**, como se muestra en la línea 2 mediante el uso de la función **type**, que obtiene la clase a la que pertenece la variable. Los guiones bajos no significan nada, son una facilidad que proporciona Python para que podamos leer mejor los números si tienen muchos dígitos.

En la línea 3 se está haciendo una conversión de tipo de la variable **numero**. La función **str** convierte el valor de la variable que se le pasa por parámetro al tipo **str**. Esta conversión se conoce como **explícita**. Existen otras funciones de conversión equivalentes para otros tipos, como **float**, **int**, **bin**, **hex**, **oct** o **bool**. Son todas parte de las funciones *built-in*. Te animo a que consultes la documentación y veas para que sirve cada una.

Ejercicio 15 – Comprobación de elementos de una lista

Enunciado:

Comprobar si el título de una serie se encuentra dentro de una tupla con varios títulos.

Solución:

```
1 series = ('The Walking Dead', 'Juego de Tronos', 'El Juego del Calamar')
2 serie_buscada = input('Introduce el título de una serie: ')
3 if serie_buscada in series:
4     print(f'La serie {serie_buscada} está en la lista')
5 else:
6     print(f'La serie {serie_buscada} no está en la lista')
7
```

Salida:

```
Introduce el título de una serie: Farmacia de Guardia
La serie Farmacia de Guardia no está en la lista
```

Explicación:

La novedad en este ejercicio es el operador **in** que se utiliza en la línea 3. Este operador sirve para preguntar si un elemento concreto está dentro de una colección de datos (tupla, lista, conjunto o las claves de un diccionario). En este caso, se trata de averiguar si el título de una serie introducida por el usuario en la línea 2 se encuentra dentro de la tupla de series definida en la línea 1.

No subestimes el operador **in**. Aunque solo tiene dos letras es muy útil. Existe la variante **not in** que sirve, como se puede deducir, para saber si un elemento no se encuentra dentro de una colección. También se utiliza para saber si una cadena está contenida dentro de otra.

Ejercicio 16 – Obtención los valores mayores y menores de una lista

Enunciado:

Dada una lista de números, obtener los elementos con mayor y menor valor.

Solución:

```
1 numeros = [8, 10, 200, -10, 250, 30, 35]
2 print(max(numeros)) # 250
3 print(min(numeros)) # -10
4
```

Explicación:

Otras dos funciones incluidas en el lenguaje: **max** y **min**. Obtienen el elemento mayor y menor respectivamente de una colección de datos. Con números funciona correctamente, pero con otro tipo de elementos habrá que indicar qué es mayor o menor: ¿Qué es mayor, un hombre de 40 años y 80 kilos de peso o una mujer de 50 años y 58 kilos?

Ejercicio 17 – Paso de parámetros por referencia

Enunciado:

¿Qué ocurre cuando se modifica una variable que contiene una lista dentro de la función que la ha recibido como argumento?

Solución:

```
1 def modificar_lista(lista):
2     lista[0] = 'Guybrush Threepwood'
3
4 personajes = ['Luigi', 'Peach', 'Ezio Auditore', 'Marcus Fenix']
5 modificar_lista(personajes)
6 print(personajes) # ['Guybrush Threepwood', 'Peach', 'Ezio Auditore', 'Marcus
7 Fenix']
```

Explicación:

Atención a este ejercicio porque muestra una característica muy relevante del lenguaje (de Python y de otros muchos). El llamado **paso de parámetros por referencia**.

La función definida en la línea 1 recibe un argumento llamado **lista**. Cuando en la línea 5 se le pasa la lista **personajes** no le está indicando el contenido de la lista sino **DONDE** se encuentra la lista. Esto significa que al modificar el elemento ubicado en la posición 0 dentro de la función, también se está modificando fuera, porque en realidad la variable **lista** de la función y la variable **personajes** de la línea 4 son la misma lista (la misma referencia).

Ejercicio 18 – Paso de parámetros por valor

Enunciado:

¿Qué ocurre cuando se modifica una variable que contiene un número dentro de la función que la ha recibido como argumento?

Solución:

```
1 def modificar_dorsal(numero_dorsal):  
2     numero_dorsal = 45  
3  
4 dorsal = 23  
5 modificar_dorsal(dorsal)  
6 print(dorsal) # 23  
7
```

Explicación:

Nos encontramos con una estructura de código similar al del ejercicio anterior, pero en este caso la modificación de la variable **numero_dorsal**, no modifica el valor de la variable **dorsal**. En este caso el paso de parámetros se hace **por valor**, o lo que es lo mismo, se copia el valor de la variable que se utiliza en la llamada en la variable que lo recoge en la función.

¿Cuál es la diferencia entre este ejercicio y el anterior? En este caso, la variable contiene un tipo básico, un entero. Y con las variables de tipos básicos como enteros, booleanos o cadenas de caracteres, el paso se hace por valor mientras que los tipos más sofisticados como las listas, los diccionarios o las clases definidas por los programadores el paso se hace por referencia.

Ejercicio 19 – Generación de listas mediante *list comprehension*

Enunciado:

Crear una lista con las palabras contenidas en otra lista convertidas a mayúsculas.

Solución:

```
1 lenguajes_programacion = ['python', 'c++', 'java', 'php', 'javascript', 'c#']
2 lenguajes_mayusculas = [lenguaje.upper() for lenguaje in lenguajes_programacion]
3 for lenguaje in lenguajes_mayusculas:
4     print(lenguaje)
5
```

Salida:

```
PYTHON
C++
JAVA
PHP
JAVASCRIPT
C#
```

Explicación:

Esta herramienta es, en mi opinión, de las más bonitas de Python. Permite crear una lista a partir de los datos una colección de datos iterable, como por ejemplo una tupla u otra lista. Lo interesante de esta herramienta es que el código es muy compacto.

En la línea 2 se muestra un ejemplo. En la parte derecha de la asignación se encuentra una *comprensión de lista* (es la traducción que he encontrado, no es mía).

La comprensión de listas se delimita con corchetes. En su forma más sencilla, está formada por un bucle **for** que recorre los elementos del iterable, en este caso la lista **lenguajes_programacion**. A la izquierda del bucle, se encuentra el elemento que se va a añadir a la nueva lista, en este caso la expresión **lenguaje.upper()** o, lo que es lo mismo, la versión en mayúsculas de cada elemento de la lista original, identificado como **lenguaje**.

El código equivalente a esta línea 2 en una programación tradicional sería el siguiente:

```
lenguajes_mayusculas = list()
for lenguaje in lenguajes_programacion:
    lenguajes_mayusculas.append(lenguaje.upper())
```

El ahorro de código es evidente.

Aunque no se utiliza en este ejercicio, la *comprensión de listas* admite sentencias **if** y **else** para condicionar la inclusión de los elementos de la nueva lista. Más adelante veremos algún ejemplo.

Mola.

Ejercicio 20 – Expresión ternaria

Enunciado:

Comprobar si una cadena contiene una determinada letra utilizando una expresión ternaria.

Solución:

```
1 palabra = 'azabache'
2 letra = 'z'
3 resultado = 'contiene' if letra.upper() \
4     in palabra.upper() else 'no contiene'
5 print(f'La palabra {palabra} {resultado} la letra {letra}')
```

Salida:

```
La palabra azabache contiene la letra z
```

Explicación:

Una expresión ternaria es el equivalente compacto a una asignación de valor utilizando sentencias **if-else**.

En la solución de este ejercicio se puede observar una expresión ternaria en las líneas 3 y 4 (las he dividido para que se lean mejor).

La explicación es sencilla: la variable **resultado** va a recibir el valor **'contiene'** si la cadena contenida en la variable **letra** convertida a mayúsculas (eso es lo que hace el método **upper()** de la clase **str**) está contenida en la cadena **palabra** convertida a mayúsculas. Si esta condición no se cumple, la variable **resultado** recibirá el valor **'no contiene'**.

Presta atención a la salida. La letra **z** aparece en minúscula, pese a que hemos utilizado el método **upper** sobre ella. Las cadenas de caracteres son **inmutables** y nunca se modifican. Los métodos generan nuevas cadenas.

Ejercicio 21 – Obtención de una lista con los elementos de una tupla que cumplen una determinada condición.

Enunciado:

Crear una lista con los elementos de una tupla de palabras que tienen determinada letra utilizando *list comprehension*.

Solución:

```
1 provincias = ('Zamora', 'Salamanca', 'Cáceres', 'Zaragoza')
2 provincias_con_z = [provincia for provincia in provincias if 'z' in provincia.lower()]
3 print(provincias_con_z) # ['Zamora', 'Zaragoza']
4
```

Explicación:

Volvemos a la *comprensión de listas*. En esta ocasión utilizando una condición para determinar si un elemento forma o no parte de la nueva lista.

En la línea 2 estamos creando una nueva lista con el nombre **provincias_con_z**. Esta lista se va a construir a partir de los elementos de la lista **provincias**, que son recorridos con un bucle **for**. La novedad es que después del bucle se añade una condición, que en este caso es que la letra 'z' se encuentre en el nombre de la provincia convertido a minúsculas (para que encuentre también las letras z mayúsculas).

Las comprensiones de listas también admiten sentencias **else**, aunque la sintaxis cambio ligeramente.

Ejercicio 22 – Ordenar los elementos de una tupla (ERROR)

Enunciado:

Ordenar los elementos de una tupla utilizando el método **sort**.

Solución:

```
1 numeros = (10, 20, -10, 40, 0, 30)
2 numeros.sort()
3 print(numeros)
4
```

Salida:

```
AttributeError: 'tuple' object has no attribute 'sort'
```

Explicación:

Este ejercicio no tiene solución, pero lo he introducido como refuerzo, ya que es importante recordar que LAS TUPLAS NO SE PUEDEN MODIFICAR.

El método **sort** de las listas, ordena los elementos de la lista sobre la que se aplica. Si la tupla no se puede modificar, es lógico pensar que no puede tener un método que modifique su contenido ¿verdad? Por lo tanto, las tuplas no tienen método **sort**.

Entonces ¿no se pueden ordenar los elementos de una tupla? En el siguiente ejercicio tendrás la respuesta.

Ejercicio 23 – Ordenar los elementos de una tupla (CORRECTO)

Enunciado:

Ordenar los elementos de una tupla utilizando la función **sorted**.

Solución:

```
1 numeros = (10, 20, -10, 40, 0, 30)
2 numeros_ordenados = sorted(numeros)
3 print(numeros_ordenados) # [-10, 0, 10, 20, 30, 40]
4
```

Explicación:

(Continúa del ejercicio anterior)

Los elementos de una tupla en realidad no se pueden ordenar dentro de la propia tupla, pero se pueden ordenar para crear una nueva estructura de datos. En este caso una lista.

La función **sorted** crea una lista a partir de, principalmente, una lista o una tupla. Por lo tanto, en la línea 2, se está pasando la tupla **numeros** como argumento a la función **sorted** para que nos proporcione una lista con los elementos ordenados que asignamos a la variable **numeros_ordenados**.

Ejercicio 24 – Funciones que reciben muchos argumentos

Enunciado:

Crear una función que recibe muchos argumentos. Tantos, que no se sabe ni cuantos son.

Solución:

```
1 def obtener_maximo(*args):
2     print(type(args)) # <class 'tuple'>
3     return max(args)
4
5 print(obtener_maximo(-5, 10, 3, 1)) # 10
6 print(obtener_maximo(80, 20)) # 80
7 print(obtener_maximo(5)) # 5
```

Explicación:

Aunque no es una situación habitual, en ocasiones una función debe recibir un número indeterminado de argumentos sin utilizar listas o tuplas como contenedores de estos.

Por ejemplo, la función **print** utiliza esta capacidad. Fíjate en la siguiente imagen:

```
print(*objects, sep=' ', end='\n', file=None, flush=False)
```

Está obtenida de la documentación de Python en la web. Nos indica que la función **print** puede recibir entre 0 y *n* objetos (***objects**) que serán los **argumentos posicionales** (los que no tienen nombre, también conocidos como **non-keyword**). Son los argumentos que se mostrarán por pantalla.

El resto de los argumentos (**sep**, **end**, **file** y **flush**) son del tipo **keyword** y hay que identificarlos en la llamada a la función. Como tienen valor por defecto, son opcionales.

De esta forma, la llamada a la función **print** podría tener las formas siguientes:

```
print()
print('hola')
print('hola',23)
print('hola',23,'estadística')
```

Los argumentos de todas estas llamadas, excepto de la primera que no tiene, se recogen en ***objects**. Es el asterisco el que indica que esa variable debe recoger todos los argumentos posicionales que se reciban.

En la solución del ejercicio, en la línea 1 se define una función que recibe ***args** que es la notación que se utiliza habitualmente, aunque el nombre puede ser el que quieras ya que lo importante es el asterisco. Internamente a la función **args** será una tupla, con todos los argumentos posicionales de la llamada.

En las líneas 5, 6 y 7. Se ven distintos ejemplos de llamada. Todos esos argumentos se recogerán en ***args** que será una tupla, como se puede observar en la línea 2.

La línea 3 retorna el valor máximo de los argumentos recibidos obtenido utilizando la función **max**, que calcula el valor máximo de un iterable.

Ejercicio 25 – Adivina, adivinanza

Enunciado:

Crear un programa que determine si el usuario es un adivino capaz de averiguar un número aleatorio creado por el propio programa.

Solución:

```
1 import random
2
3 UMBRAL_INTENTOS = 3
4 numero_aleatorio = random.randint(1, 10)
5 numero_candidato = 0
6 numero_intentos = 0
7 while (numero_candidato!=numero_aleatorio):
8     numero_intentos += 1
9     numero_candidato = int(input('Introduce un número: '))
10    if (numero_candidato==numero_aleatorio):
11        print('¡Has acertado!')
12    else:
13        print('¡Inténtalo de nuevo!')
14 if (numero_intentos>UMBRAL_INTENTOS):
15     print('No eres muy buen adivino que digamos...')
16 else:
17     print('¡Eres un adivino!')
```

Salida:

```
Introduce un número: 7
¡Inténtalo de nuevo!
Introduce un número: 5
¡Inténtalo de nuevo!
Introduce un número: 6
¡Inténtalo de nuevo!
Introduce un número: 8
¡Inténtalo de nuevo!
Introduce un número: 4
¡Inténtalo de nuevo!
Introduce un número: 2
¡Inténtalo de nuevo!
```

```
Introduce un número: 1
¡Has acertado!
No eres muy buen adivino que digamos...
```

Explicación:

Este es uno de mis ejercicios favoritos en los cursos de iniciación a Python. Para los programadores experimentados no es muy complejo, pero para aquellas personas que se estén iniciando en este sufrido mundo de la programación puede resultar muy interesante. Vamos a explicar la solución.

En la línea 1 se importa el módulo **random** de Python. Este módulo proporciona herramientas para la generación de números pseudo-aleatorios.

En la línea 3 se define una "constante". En Python NO hay constantes, pero hacemos como si las hubiese, nombrando a la variable con letras mayúsculas. Todos los programadores sabemos que esta notación se corresponde con una constante y actuamos en consecuencia.

En la línea 4 se genera un número entero aleatorio entre los números 1 y 10, ambos incluidos con la función **randint**.

En la línea 7 aparece un bucle **while**, cuyo contenido se ejecutará mientras que la condición indicada entre paréntesis se cumpla. Esta condición dice que mientras que el número candidato introducido por el adivino sea distinto del número aleatorio que tiene que encontrar, repita el contenido del bucle.

Como el bucle **while** primero hace la comprobación de la condición, en este caso tenemos que asegurar que se va a entrar dentro al menos una vez. Para ello inicializamos la variable **numero_candidato** a 0, que es un valor que no se encuentra dentro del rango del número (pseudo) aleatorio generado.

El resto del código es fácil de entender. Así lo espero. Ahora te toca comprobar si eres un buen adivino. Yo he necesitado 7 intentos para averiguar un número entre 1 y 10. No digo más.

Ejercicio 26 – Escritura de datos en un fichero

Enunciado:

Pedir el nombre y la ciudad de nacimiento al usuario y escribir los datos en un fichero.

Solución:

```
1 NOMBRE_FICHERO = 'fichero_e26.txt'
2
3 nombre = input('Introduce tu nombre: ')
4 ciudad_nacimiento = input('Introduce tu ciudad de nacimiento: ')
5
6 fichero = open(NOMBRE_FICHERO, mode='wt')
7 fichero.write(nombre + '\n')
8 fichero.write(ciudad_nacimiento + '\n')
9 fichero.close()
```

Explicación:

Muchos programas necesitan almacenar algunos datos para que estos no se pierdan al terminar su ejecución, por lo que tenemos que conseguir que dichos datos sean **persistentes**. Una de las maneras más sencillas de hacer esto es a través de ficheros.

En la línea 6 se utiliza la función **open** para abrir un fichero en modo “write-text” (eso significa la cadena **'wt'** asignada al argumento **mode**).

Las líneas 7 y 8 escriben en el fichero (a través de la variable **fichero**) el nombre y la ciudad de nacimiento. En ambos casos se añade al final del texto un salto de línea (“\n”).

Por último, en la línea 9, se cierra el fichero mediante la llamada al método **close**. Es imprescindible dejar el fichero cerrado para evitar errores posteriores.

Ejercicio 27 – Lectura de datos de un fichero

Enunciado:

Leer el nombre y la ciudad de nacimiento de un fichero de datos (el que hemos creado en el ejercicio anterior).

Solución:

```
1 NOMBRE_FICHERO = 'fichero_e26.txt'
2
3 fichero = open(NOMBRE_FICHERO, 'rt')
4 nombre = fichero.readline().replace('\n', '')
5 ciudad_nacimiento = fichero.readline().replace('\n', '')
6 fichero.close()
7
8 print('Nombre:', nombre) # Nombre: Fernando
9 print('Ciudad de nacimiento:', ciudad_nacimiento) # Ciudad de nacimiento: Madrid
```

Explicación:

Para poder leer de un fichero hace falta abrirlo en algún modo que permite realizar operaciones de lectura. En este caso el modo utilizado es **'rt'** (*"read-text"*) pero no indicamos el nombre del argumento, haciendo una referencia posicional.

En las líneas 4 y 5, se están leyendo las dos primeras líneas del fichero respectivamente, utilizando el método **readline** de fichero. Este método devuelve la línea completa en un objeto **str** (una cadena de caracteres), incluyendo el salto de línea. Como dicho salto de línea no nos interesa, lo eliminamos haciendo uso del método **replace** de la clase **str** diciendo que queremos eliminar la cadena **'\n'** (salto de línea) por una cadena vacía.

Una vez procesado el fichero sólo nos quedaría cerrarlo (línea 6) y mostrar los datos por pantalla.

Ejercicio 28 – Creación de un fichero de configuración utilizando JSON

Enunciado:

Crear un fichero de configuración que almacene algunos parámetros de configuración de una aplicación. Utilizar el formato JSON.

Solución:

```
1 import json
2
3 def escribir_parametros(parametros : dict, nombre_fichero : str):
4     with open(nombre_fichero, 'w') as fichero:
5         json.dump(parametros, fichero)
6
7
8 COLOR_FONDO = '#FFFFFF'
9 COLOR_TEXTO = '#000000'
10 IDIOMA = 'ESPAÑOL'
11 NOMBRE_FICHERO = 'parametros_e28.json'
12
13 parametros = dict()
14 parametros['color_fondo'] = COLOR_FONDO
15 parametros['color_texto'] = COLOR_TEXTO
16 parametros['idioma'] = IDIOMA
17 escribir_parametros(parametros, NOMBRE_FICHERO)
18
```

Salida:

JSON	Datos sin procesar	
Guardar	Copiar	Contraer todo
color_fondo:	"#FFFFFF"	
color_texto:	"#000000"	
idioma:	"ESPAÑOL"	

Explicación:

Un fichero que contiene un documento en formato JSON es ideal para almacenar la configuración de una aplicación. Si no conoces el formato JSON, busca información en la web. Tiene una sintaxis muy sencilla y no debería resultarte difícil de entender.

El módulo **json** importado en la línea 1 proporciona utilidades para trabajar con documentos JSON. Una de ellas es la capacidad de almacenar un diccionario expresado como un documento JSON en un fichero a través de la función **dump**.

En la línea 5 se utiliza **dump** para escribir el diccionario recibido como argumento de la función **escribir_parametros** en un fichero abierto en la línea 4 con la sentencia **with**.

La sentencia **with** permite abrir un fichero que será cerrado automáticamente al finalizar el bloque definido a continuación. En este caso, la línea 4 realiza la apertura del fichero y abre un bloque de código. La línea 5 (debidamente sangrada para que esté dentro de dicho bloque) hace uso de la función **dump** para escribir el diccionario **parametros** en el fichero. Al finalizar el bloque, el fichero se cerrará automáticamente.

El resto del programa está compuesto por la declaración de una serie de constantes, la creación de un diccionario con un conjunto de claves y valores y la invocación a la función **escribir_parametros** para que almacene dicho diccionario en un fichero en formato json.

En la salida se puede observar el contenido del fichero generado utilizando el navegador Firefox, que es capaz de representar los documentos JSON con un formato que facilita su legibilidad.

Ejercicio 29 – Lectura de un fichero de configuración en formato **JSON**

Enunciado:

Leer el fichero de configuración almacenado en el ejercicio anterior.

Solución:

```
1 import json
2
3 def leer_parametros(nombre_fichero : str):
4     with open(nombre_fichero, 'r') as fichero:
5         datos = json.load(fichero)
6     return datos
7
8 NOMBRE_FICHERO = 'parametros_e28.json'
9
10 parametros = leer_parametros(NOMBRE_FICHERO)
11
12 COLOR_FONDO = parametros['color_fondo']
13 COLOR_TEXTO = parametros['color_texto']
14 IDIOMA = parametros['idioma']
15 print(COLOR_FONDO) # #FFFFFF
16 print(COLOR_TEXTO) # #000000
17 print(IDIOMA) # ESPAÑOL
```

Explicación:

Como se podía esperar, después de escribir un fichero se deberá leer para recuperar los datos almacenados en este.

De nuevo hacemos uso del módulo **json**. En este caso, utilizamos la función **load** que nos permite leer un fichero en formato JSON y almacenar los datos

contenidos en este en un diccionario. Es tan sencillo como se muestra en la línea 5.

El proceso completo que se muestra en este ejercicio es el siguiente:

- Importación del módulo **json** (línea 1)
- Definición de la función que lee un fichero JSON y devuelve un diccionario con su contenido (líneas 3 a 6).
- Definición de una “constante” con el nombre del fichero (línea 8).
- Llamada a la función de lectura y almacenamiento del resultado en una variable de tipo **dict** (línea 10).
- Acceso a los elementos del diccionario para obtener los valores y asignárselos a unas “constantes” (líneas 12 a 14).
- Se muestra el valor de las constantes (líneas 15 a 17).

Habríamos recuperado la configuración almacenada en el ejercicio anterior, completando el proceso de almacenamiento y recuperación de parámetros utilizando JSON.

Ejercicio 30 – Calculadora

Enunciado:

Crear una calculadora utilizando funciones.

Solución:

```
1 def sumar(a, b):
2     return a + b
3
4 def restar(a, b):
5     return a - b
6
7 def multiplicar(a, b):
8     return a * b
9
10 def dividir(a, b):
11     if (b!=0):
12         return a / b
13     else:
14         raise ArithmeticError('No se puede dividir entre 0')
15
16 def elevar(a, b):
17     return a**b
18
19 print(sumar(3,8)) # 11
20 print(restar(3,8)) # -5
21 print(multiplicar(3,8)) # 24
22 print(dividir(3,8)) # 0.375
23 print(elevar(3,8)) # 6561
24 print(sumar('imagen','txt')) # imagentxt
25 print(multiplicar('imagen',3)) # imagenimagenimagen
26
```

Explicación:

Vamos a analizar la solución. Hemos definido 5 funciones que realizan las operaciones aritméticas sumar, restar, multiplicar, dividir y elevar a una potencia, siendo **dividir** la función distinta.

Esta función devuelve el resultado de la división si el divisor (argumento **b**) es distinto de cero. Si fuese cero ejecuta la siguiente línea de código:

```
raise ArithmeticError('No se puede dividir entre 0')
```

En esta línea se está construyendo un objeto de la clase **ArithmeticError** (una de las *built-in exceptions* o excepciones integradas en Python) y se propaga utilizando la instrucción **raise**. Esto provocará un error si se hace una llamada a dividir pasándole un 0 como valor al argumento **b** (el divisor).

En la siguiente imagen generada por una IA se puede ver a un oso enfadado lanzando (**raise**) una pelota (**ArithmeticException**) porque le han pedido que divida algo entre cero.



En realidad, Python ya genera por sí mismo este error cuando se intenta dividir entre 0, pero creo que como ejemplo es ilustrativo y fácil de comprender.

El resto de las líneas de código (19 a 25) son llamadas a las funciones. Incorpora una nueva llamada a la función **dividir** indicando un 0 como divisor y así puedes ver como “explota” el programa.

Ejercicio 31 – El módulo **math**

Enunciado:

Utilizar algunas de las múltiples funciones y constantes que incluye el módulo **math**.

Solución:

```
1 import math
2
3 print(math.pi) # 3.141592653589793
4 print(math.e) # 2.718281828459045
5 print(math.cos(3.2)) # -0.9982947757947531
6 print(math.sin(3.2)) # -0.058374143427580086
7 print(math.dist([3,2],[10,2])) # 7.0
8
```

Explicación:

En este ejercicio únicamente presentamos el módulo **math** de Python para darlo a conocer en sociedad.

Este módulo incorpora constantes y funciones matemáticas, como se puede deducir de su nombre.

En la solución (es más bien un ejemplo) se hacen uso de las constantes **pi** y **e**, así como de las funciones **cos** (coseno), **sin** (seno) y **dist** (distancia euclidiana entre dos puntos).

Si las matemáticas forman parte de tu vida, este es tu módulo (al menos uno de ellos). Echa un vistazo a la documentación de Python para que puedas ver toda la funcionalidad que proporciona.

Ejercicio 32 – En busca del par perdido: la sentencias **break** y **else**

Enunciado:

Recorrer una lista de números con un bucle **for** buscando el primer número par.

Solución:

```
1 numeros = [7, 1, 3, 8, 10, 6, 9, 5]
2
3 for numero in numeros:
4     if numero%2==0:
5         print(f'El número {numero} es par')
6         break
7 else:
8     print('No hay ningún número par en la lista')
9
```

Salida:

```
El número 8 es par
```

Explicación:

Un bucle **for** sirve, entre otras cosas, para recorrer un objeto **iterable**, como las listas o las tuplas.

El inconveniente surge cuando recorremos un iterable buscando el primer elemento que cumple con una determinada condición y lo encontramos. ¿Hay que seguir recorriendo y analizando el resto de los elementos del objeto iterable, aunque ya no sirva para nada? La respuesta es no, ya que utilizando la sentencia **break** (línea 6) se interrumpe la ejecución del bucle.

La novedad en Python frente a otros lenguajes es que los bucles **for** admiten la sentencia **else**.

La idea es realmente buena, porque a veces se necesita saber si el bucle ha terminado de forma anticipada (por ejemplo, hemos encontrado lo que buscábamos) o ha finalizado de recorrer el objeto iterable del todo. El bloque asociado a **else** se ejecutará cuando el bucle se haya completado o, lo que es lo mismo, cuando no se haya utilizado **break** como forma de interrumpir su ejecución.

Si conoces otros lenguajes de programación piensa en cómo habría que programar este comportamiento y puede que llegues a la misma conclusión que yo: es una buena idea.

Ejercicio 33 – Paso de todo. La sentencia **pass**.

Enunciado:

Utilizar la sentencia **pass** para no hacer nada.

Solución:

```
1 importe = 1_000_000
2
3 if importe>1_000:
4     pass
5 elif importe>100:
6     pass
7 elif importe>10:
8     pass
9 else:
10     pass
11
```

Explicación:

¿No quiero hacer nada, pero quiero el programa esté libre de errores sintácticos en determinados bloques de código? Mala idea, porque Python obliga a programar algo en dichos bloques. Por ejemplo, en una clase, en una función, en una sentencia **if** o en un bucle **for**, estamos obligados a programar el bloque de código asociado.

Existe una solución de compromiso para dar gusto a todo el mundo: la sentencia **pass**. Es una sentencia nula que no hace nada más que ocupar espacio y decirle al intérprete de Python que hay código para que no provoque un error. Fin del problema. Si estás escribiendo un programa y aún no tienes la capacidad, el tiempo o las ganas de completar un bloque de código pon la sentencia **pass** y déjalo para más adelante.

Nadie dijo que el código tenía que hacer algo. 😊

Ejercicio 34 – Destruir al monstruo con la sentencia **match**

Enunciado:

¿Cómo elegir la herramienta perfecta para acabar con el monstruo que acecha en la oscuridad? La sentencia **match** tiene la respuesta.

Solución:

```
1 def obtener_metodo_destruccion(monstruo):
2     match monstruo:
3         case 'Licántropo':
4             return 'Bala de plata'
5         case 'Zombi':
6             return 'Hacha'
7         case 'Vampiro':
8             return 'Estaca de madera'
9         case 'Charlatán':
10            return 'Paciencia'
11        case _:
12            return 'Lo siento'
13
14 print(obtener_metodo_destruccion('Vampiro'))
15
```

Salida:

```
Estaca de madera
```

Explicación:

La sentencia **match** no siempre existió en Python. Se incorporó en la versión 3.10 (año 2021). En otros lenguajes se suele llamar **switch**. Antes se hacía con sentencias **if** y **elif** o con diccionarios, pero esos tiempos ya quedaron atrás.

Nuestro querido **match** funciona de la siguiente manera:

En la línea 2 aparece la sentencia **match** junto con una variable. Lo que va a hacer esta sentencia es comparar el valor de la variable con algunas de las cadenas de caracteres asociadas a la sentencia **case**, ejecutando el bloque de código que se corresponda con la opción que ha coincidido (han hecho “match”).

Como se puede ver, en esta solución, existen 5 opciones **case**, con diferentes bloques de código. La última de ellas, indica como valor de comparación un guion bajo (**case _**). Esta es la opción por defecto y es opcional.

En resumen, la función **obtener_metodo_destruccion** recibe el nombre de un monstruo, decide qué método hay que utilizar en función de dicho nombre y lo devuelve. Si no tiene registrado el nombre del monstruo, proporciona un mensaje de solidaridad, que no sé si ayudará mucho.

En otros lenguajes de programación, cada bloque asociado a una sentencia **case** finaliza (opcionalmente) con una instrucción **break**. En Python esto no es necesario, ya que las opciones son excluyentes entre sí.

Ejercicio 35 – Vayamos por partes: el método **split**

Enunciado:

Obtener datos de una cadena que utiliza un carácter como separador de campos.

Solución:

```
1 producto = 'A14VHG-019FR#MSI#Titan 18 HX Dragon#5099'
2 campos = producto.split('#') # Obtiene una lista
3 print(f'Código: {campos[0]}. Marca: {campos[1]}. \
4 Modelo: {campos[2]}. Precio: {campos[3]}')
5
```

Salida:

```
Código: A14VHG-019FR. Marca: MSI. Modelo: Titan 18 HX
Dragon. Precio: 5099
```

Explicación:

El método **split** de la clase **str** (cadenas de caracteres) genera una lista con los diferentes pedazos que componen una cadena. ¿Cómo sabe por dónde trocear? Ahí es donde participa el delimitador que se le indica en la llamada.

En la línea 2 de este ejercicio, se invoca al método **split** sobre la cadena **producto**, indicando la cadena '#' como separador. Este método busca en la cadena **producto** el separador y va obteniendo las diferentes partes, agregándolas a una lista. El separador se descarta, claro.

Ejercicio 36 – Lectura de ficheros **csv**

Enunciado:

Leer el fichero **películas_western.csv** en formato **csv** que contiene la siguiente información:

```
"El bueno, el malo y el feo",1966,Sergio Leone
Los imperdonables,1950,John Huston
Más corazón que odio,1956,John Ford
Érase una vez en el Oeste,1968,Sergio Leone
Los siete magníficos,1960,John Sturges
Río Bravo,1959,Howard Hawks
```

Solución:

```
1 import csv
2
3 def leer_fichero_peliculas(nombre_fichero):
4     with open(nombre_fichero, 'r', encoding='utf-8') as fichero:
5         datos = csv.reader(fichero)
6         return list(datos)
7
8 películas = leer_fichero_peliculas('películas_western.csv')
9 for película in películas:
10     print(película)
```

Salida:

```
['El bueno, el malo y el feo', '1966', 'Sergio Leone']
['Los imperdonables', '1950', 'John Huston']
['Más corazón que odio', '1956', 'John Ford']
['Érase una vez en el Oeste', '1968', 'Sergio Leone']
['Los siete magníficos', '1960', 'John Sturges']
['Río Bravo', '1959', 'Howard Hawks']
```

Explicación:

El formato de fichero **csv** (*Comma Separated Values* o Valores Separados por comas) es utilizado frecuentemente para el almacenamiento de datos tabulares.

Python incorpora de manera nativa un módulo con el nombre **csv** que contiene funciones de utilidad para manejar este tipo de ficheros.

Por ejemplo, en la línea 5 se está haciendo uso de la función **reader** que toma el fichero de entrada (una sucesión de líneas cuyos campos están separados por comas) y genera una lista con campos contenidos en cada línea.

En el fichero csv de entrada utilizado en la solución del ejercicio, **peliculas_western.csv**, se puede observar que el primer campo de la primera línea está delimitado por comillas. Esto es debido a que contiene una coma y si no indicamos que esta forma parte del campo, la tomaría como un delimitador.

En muchas ocasiones los ficheros **csv** utilizan delimitadores distintos de la coma. No es un problema, ya que la función **reader** permite indicar que delimitador se quiere utilizar.

Ejercicio 37 – Aplicación gráfica con tkinter

Enunciado:

Crear una aplicación que muestre una ventana que contenga un texto y un botón con el que cerrar la propia aplicación.

Solución:

```
1 import tkinter as tk
2
3 ventana = tk.Tk()
4 ventana.title("Mi aplicación")
5 ventana.geometry("320x240")
6
7 etiqueta = tk.Label(ventana, text="¡Hola!", font=("Arial", 20))
8 etiqueta.pack(pady=50)
9
10 boton = tk.Button(ventana, text="Cerrar", command=ventana.quit)
11 boton.pack()
12
13 ventana.mainloop()
```

Salida:



Explicación:

Tkinter es una librería para hacer interfaces gráficas de usuario (GUI) que serán compatibles con cualquier sistema operativo (como todo en Python). En realidad, es un 'envoltorio' (wrapper) de **Tcl/Tk**, una combinación de lenguaje de programación (**Tcl**) y una biblioteca gráfica (**Tk**). **Tkinter** nos proporciona la capacidad de utilizar **Tcl/Tk** de forma sencilla.

En este sencillo ejercicio se está creando una ventana que contiene un botón programado y un texto.

En la línea 1, se importa la librería **tkinter** y se renombra como **tk**.

En la línea 3 se construye una instancia de la clase **Tk** que constituye la ventana principal de la aplicación. En las líneas 4 y 5 se le da a esta ventana un título y unas dimensiones.

En las líneas 7 y 10 se están creando una etiqueta y un botón, respectivamente. Como se puede suponer, **tkinter** ofrece muchos más componentes (*widgets*) como **Entry**, para permitir la inserción de datos, **ListBox** para mostrar listas o **Menu** para crear menús. Cada uno de estos elementos tiene sus propios parámetros que habrá que conocer antes de usarlos consultando la documentación.

En las líneas 8 y 11 se utiliza el método **pack** que se encarga de colocar los componentes en la ventana.

La sentencia **ventana.mainloop()** se encarga de que la ventana esté activa a la espera de eventos mientras no se decida terminar la ejecución. Esto ocurrirá como consecuencia de pulsar sobre el icono de cierre de la ventana o, en este caso, pulsar el botón, ya que está asociado al método **quit** de la ventana (vuelve a consultar la línea 10 y comprueba como se asocia al argumento **command** la acción **ventana.quit**).

Además de **Tkinter** (nativa), existen otras librerías para realizar interfaces gráficas como **PyQt**, **Kivy** o **wxPython**, por citar algunas. Si quieres hacer aplicaciones con interfaz gráfica descubre cuál es la más adecuada.

Ejercicio 38 – Programación funcional: la función **filter**

Enunciado:

Utilizar la función **filter** para obtener las películas dirigidas por **Ridley Scott**.

Solución:

```
1 blade_runner = {'titulo': 'Blade Runner', 'director': 'Ridley Scott', 'año': 1982}
2 el_resplandor = {'titulo': 'El resplandor', 'director': 'Stanley Kubrick', 'año': 1980}
3 el_padrino = {'titulo': 'El padrino', 'director': 'Francis Ford Coppola', 'año': 1972}
4 alien = {'titulo': 'Alien', 'director': 'Ridley Scott', 'año': 1979}
5
6 peliculas = (blade_runner, el_resplandor, el_padrino, alien)
7
8 def es_director(pelicula):
9     return pelicula['director'].upper() == 'RIDLEY SCOTT'
10
11 pelis_ridley = filter(es_director, peliculas)
12
13 print(list(pelis_ridley))
14
```

Salida:

```
[{'titulo': 'Blade Runner', 'director': 'Ridley Scott', 'año': 1982}, {'titulo': 'Alien', 'director': 'Ridley Scott', 'año': 1979}]
```

Explicación:

La función **filter** permite obtener el subconjunto de elementos de un iterable (lista, tupla...) que cumplen una condición y almacenarlo en un **iterador** (un objeto que contiene múltiples elementos que se pueden recorrer) .

En la línea 8 se define una función que recibe una película (un diccionario) y devuelve **True** o **False** si el director es **RIDLEY SCOTT** (el de Alien).

En la línea 11 se utiliza **filter** pasándole como argumentos la función de la línea 8 y una tupla con películas. El resto es historia del cine.

Ejercicio 39 – Programación funcional: la función **map**

Enunciado:

Convertir los datos de una serie de películas almacenadas en tuplas en diccionarios mediante la función **map**.

Solución:

```
1 datos_1 = ('Batman', 'Tim Burton', 1989)
2 datos_2 = ('Hulk', 'Ang Lee', 2003)
3 datos_3 = ('Superman', 'Richard Donner', 1978)
4
5 datos = (datos_1, datos_2, datos_3)
6
7 def crear_pelicula(datos):
8     return {'titulo':datos[0], 'director':datos[1],
9           'año':datos[2]}
10 peliculas = map(crear_pelicula, datos)
11
12 print(list(peliculas))
13
```

Salida:

```
[{'titulo': 'Batman', 'director': 'Tim Burton', 'año': 1989}, {'titulo': 'Hulk', 'director': 'Ang Lee', 'año': 2003}, {'titulo': 'Superman', 'director': 'Richard Donner', 'año': 1978}]
```

Explicación:

Esta función permite aplicar una función a todos los elementos de un iterable, devolviendo el resultado en una estructura de datos. En la línea 10, se aplica la función **crear_pelicula** a todos los elementos de la tupla **datos**. El resultado es un **iterador**.

Ejercicio 40 – Programación funcional: la función **reduce**

Enunciado:

Obtener el acumulado de una colección de datos mediante el uso de la función **reduce**.

Solución:

```
1 from functools import reduce
2
3 datos_trimestre_1 = (800, 1200)
4 datos_trimestre_2 = (400, 900)
5 datos_trimestre_3 = (500, 1000)
6 datos_trimestre_4 = (100, 200)
7
8 datos_anuales = (datos_trimestre_1, datos_trimestre_2,
9                  datos_trimestre_3, datos_trimestre_4)
10
11 def obtener_ingresos(acumulado, datos_trimestrales):
12     print(f'Acumulado:{acumulado}', end='. ')
13     print(f'Ingresos mensuales:{datos_trimestrales[1]}')
14     return acumulado+datos_trimestrales[1]
15
16 ingresos = reduce(obtener_ingresos, datos_anuales, 0)
17
18 print(f'Ingresos totales:{ingresos}')
19
```

Explicación:

La función **reduce** es lo que hace un robot de cocina: metes un montón de ingredientes y te devuelve una croqueta. No es del todo así, pero casi.

Esta función recibe un iterable, que no es otra cosa que un conjunto de elementos, y devuelve un único objeto, del tipo que sea.

Por ejemplo, a partir de un conjunto de clientes entre cuyos datos se encuentra la facturación anual, obtener el acumulado de los ingresos anuales de todos los clientes.

En la línea 16 de este ejercicio se ejecuta la función **reduce** que recibe como argumentos una función (**obtener_ingresos**), una tupla principal (**datos_anuales**) que contiene varias tuplas secundarias (**datos_trimestre_x**) y un valor inicial (0 en este caso).

Cada tupla secundaria está compuesta por dos cifras, siendo la primera los gastos y la segunda los ingresos trimestrales.

La función **obtener_ingresos** recibirá cada elemento de la tupla y el acumulado hasta entonces, sumando la segunda cifra de cada tupla secundaria (los gastos) hasta obtener la suma de los ingresos.

La salida genera por el programa es la siguiente:

```
Acumulado:0. Ingresos mensuales:1200
Acumulado:1200. Ingresos mensuales:900
Acumulado:2100. Ingresos mensuales:1000
Acumulado:3100. Ingresos mensuales:200
Ingresos totales:3300
```

Ejercicio 41 –Programación orientada a objetos: definición de clase

Enunciado:

Creación de una clase que represente un videojuego, definición de atributos y programación de métodos.

Solución:

```
1 class Videojuego:
2     def __init__(self, titulo, genero, plataforma):
3         self.titulo = titulo
4         self.genero = genero
5         self.plataforma = plataforma
6
7     def es_multiplataforma(self):
8         return self.plataforma == 'Multiplataforma'
9
10    def escribir_datos(self):
11        fichero = open(f'{self.titulo.lower()}.txt', mode='w', encoding='utf-8')
12        fichero.write(f'Título: {self.titulo}\n')
13        fichero.write(f'Género: {self.genero}\n')
14        fichero.write(f'Plataforma: {self.plataforma}\n')
15
```

Explicación:

Python admite la orientación a objetos como paradigma de programación. Si no sabes nada de programación orientada a objetos te vendría bien conocer los fundamentos, aunque ya te aviso de que llegar de 0 a infinito en un fin de semana en esta área de la programación no es muy realista. Vayamos paso a paso, comenzando por la definición de una clase.

En este ejercicio se está construyendo una clase que representa a un videojuego.

En la línea 1 aparece la definición de la clase. Se utiliza la palabra **class** seguida del nombre de la clase. La convención de nombres dice que la

primera letra de cada palabra que forma el nombre de la clase debe estar escrita en mayúsculas y el resto en minúsculas.

En la línea 2 se define un método especial con el nombre `__init__`. Este método es el que se encarga de inicializar los objetos de la clase. Recibe como argumentos la palabra **self** (es una convención) que contiene una referencia al objeto y a continuación los argumentos que se necesiten para inicializar el objeto.

En las líneas 3, 4 y 5 se crean e inicializan los atributos, que son las variables que caracterizan a los objetos. Van precedidos de la palabra **self**, que es la referencia al objeto al que pertenecen.

En la línea 7 se define un método. La notación y la estructura es la misma que la de las funciones, salvo que debe recibir como argumento **self** (insisto en que es una convención, podría llamarse de cualquier otra manera). En el código del método, en la línea 8, se devuelve el resultado de comparar el valor del atributo **self.plataforma** con el literal **'Multiplataforma'**.

El método **escribir_datos**, definido en la línea 10, crea un fichero con los valores de los atributos.

Todas las referencias a atributos de instancia de un objeto deben ir precedidos de **self** dentro de la clase. Es una forma de decir “voy a utilizar este atributo de mí mismo”. Por eso cada vez que aparece un atributo en el código va precedido de **self**. Es necesario para diferenciar atributos de variables.

La ejecución de este ejercicio no tiene ningún efecto. Únicamente se está creando la clase, pero no se está utilizando. En el siguiente ejercicio ya veremos en acción nuestro trabajo.

Ejercicio 42 – Programación orientada a objetos: instanciación de objetos e invocación de métodos

Enunciado:

A partir de la solución del ejercicio anterior, crear una instancia de la clase **Videojuego** y utilizar sus métodos.

Solución:

```
1 class Videojuego:
2     def __init__(self, titulo, genero, plataforma):
3         self.titulo = titulo
4         self.genero = genero
5         self.plataforma = plataforma
6
7     def es_multiplataforma(self):
8         return self.plataforma == 'Multiplataforma'
9
10    def escribir_datos(self):
11        fichero = open(f'{self.titulo.lower()}.txt', mode='w', encoding='utf-8')
12        fichero.write(f'Título: {self.titulo}\n')
13        fichero.write(f'Género: {self.genero}\n')
14        fichero.write(f'Plataforma: {self.plataforma}\n')
15
16    minecraft = Videojuego('Minecraft', 'Sandbox', 'Multiplataforma')
17    print(minecraft.es_multiplataforma()) # True
18    minecraft.escribir_datos()
19
```

Salida:

Se genera el fichero **minecraft.txt** con el siguiente contenido:

```
Título: Minecraft
Género: Sandbox
Plataforma: Multiplataforma
```

Explicación:

Una vez que se define una clase hay que crear objetos. A este proceso se le conoce como instanciación.

Para crear una instancia de una clase en Python, hay que indicar el nombre de la clase y pasar como argumentos los parámetros definidos en el método `__init__`, que como se puede deducir de su nombre, es el encargado de inicializar el objeto.

En la línea 16 se puede observar la instanciación del objeto de la clase **Videojuego**. También se asocia la instancia una variable, en este caso llamada **minecraft**. Esta asociación lo que proporciona es una referencia al objeto, necesaria para poder utilizarlo posteriormente.

En las líneas 17 y 18 se utiliza la referencia al objeto para ejecutar sus métodos. En la programación orientada a objetos, cualquier interacción sobre un objeto debe ir precedida de la referencia a este. Por ejemplo, la sentencia **minecraft.escribir_datos()** de la línea 18 está diciendo al objeto **minecraft** que ejecute su método **escribir_datos**. De la misma manera, en la línea 8, se puede como se utiliza **self.plataforma** para hacer referencia al atributo **plataforma** del objeto **self**, que es como se llaman a sí mismos los objetos.

Ejercicio 43 –Métodos mágicos `__str__` y `__repr__`

Enunciado:

Sobrescribir los métodos mágicos `__str__` y `__repr__` para poder mostrar la información contenida en las instancias de una clase.

Solución:

```
1 class Impresora:
2     def __init__(self, marca, modelo, pvp):
3         self.marca = marca
4         self.modelo = modelo
5         self.pvp = pvp
6
7     def __str__(self):
8         return f'Marca: {self.marca}, Modelo: {self.modelo}, PVP: {self.pvp}'
9
10    def __repr__(self):
11        return f'Impresora({self.marca}, {self.modelo}, {self.pvp})'
12
13    canon_pixma = Impresora('Canon', 'Pixma MG3650S', 59)
14    hp_deskjet = Impresora('HP', '2822e', 49.90)
15    impresoras = (canon_pixma, hp_deskjet)
16
17    print(canon_pixma) # Marca: Canon, Modelo: Pixma MG3650S, PVP: 59
18    print(hp_deskjet) # Marca: HP, Modelo: 2822e, PVP: 49.9
19    print(impresoras) # (Impresora(Canon, Pixma MG3650S, 59), Impresora(HP, 2822e,
20    49.9))
```

Explicación:

A falta de una herencia explícita (si no sabes que es la herencia, investiga un poco para tener una idea básica) todas las clases de Python heredan de una clase denominada **object**. Esta herencia no se tiene por qué reflejar en el código, pero tiene muchos efectos. Uno de hechos es que cualquier clase tiene una buena lista de métodos “mágicos” destinados a dar determinada funcionalidad a la clase. Los nombres de estos métodos van rodeados de doble guion bajo a ambos lados. Un ejemplo es el método `__init__`.

Entre los métodos mágicos de uso más habitual están los métodos `__str__` y `__repr__`.

A veces se quiere ver la representación en forma de cadena de caracteres de un objeto. Si el objeto es de Python (un entero, una cadena o un booleano), funcionará correctamente, pero si el objeto es de una clase que hemos creado nosotros, al hacer pasárselo como argumento a la función **print** mostrará algo raro del tipo siguiente:

```
<__main__.MiClase object at 0x000001B7165786E0>
```

Da información del módulo, la clase y de la posición de memoria, pero no muestra lo que contiene el objeto que es lo que normalmente vamos a querer ver.

El método **__str__** es el que determina la representación “amigable” en formato *string* de un objeto, por lo que hay que hacer que devuelva aquello que queramos que se vea cuando se utiliza como argumento de una llamada a **print** como se puede ver en la línea 17.

Por su parte, el método **__repr__** hace algo parecido, pero en otro contexto. Debe devolver una representación más precisa del objeto. Por ejemplo, cuando se pasa una lista como argumento a una llamada a la función **print** se ejecutará este método.

Se puede simplificar la diferencia diciendo que **__str__** sirve para generar una representación para los humanos y **__repr__** una para los desarrolladores (que también somos humanos). En ambos casos, los métodos deben devolver una cadena de caracteres.

Ejercicio 44 – Programación orientada a objetos: *list-comprehension*

Enunciado:

Crear mediante *list-comprehension* una lista con las descripciones de una tupla de objetos de tipo **Receta**.

Solución:

```
1 class Receta:
2     def __init__(self, nombre, ingredientes, tiempo):
3         self.nombre = nombre
4         self.ingredientes = ingredientes
5         self.tiempo = tiempo
6
7     def obtener_descripcion(self):
8         descripcion = \
9             f"Receta de {self.nombre} con los \
10 siguientes ingredientes: {self.ingredientes}. \
11 Tiempo de preparación: {self.tiempo} minutos."
12         return descripcion
13
14 pollo_al_ajillo = \
15     Receta('Pollo al ajillo',
16           ('Pollo', 'Ajo', 'Aceite', 'Sal'), 40)
17 huevo_frito = \
18     Receta('Huevo frito',
19           ('Huevo', 'Aceite', 'Sal'), 5)
20
21 recetas = (pollo_al_ajillo, huevo_frito)
22 descripciones = \
23     [receta.obtener_descripcion() for receta in recetas]
24
25 print(descripciones)
26
```

Salida:

```
[ "Receta de Pollo al ajillo con los siguientes  
ingredientes: ('Pollo', 'Ajo', 'Aceite', 'Sal'). Tiempo  
de preparación: 40 minutos.", "Receta de Huevo frito con  
los siguientes ingredientes: ('Huevo', 'Aceite', 'Sal').  
Tiempo de preparación: 5 minutos." ]
```

Explicación:

La comprensión de listas (*list comprehension*) es una de las estructuras programáticas de Python que más me gustan. Permite crear una lista a partir de una secuencia (una lista, una tupla o cualquier estructura iterable) en una única línea.

La sintaxis básica es la siguiente:

```
nueva_lista = [nuevo_item for item in iterable]
```

En las líneas 22 y 23 se puede observar un uso de la comprensión de listas. Se está creando una nueva lista llamada **descripciones** que va a estar compuesta por todas las descripciones de cada **receta** de la lista **recetas**. La explicación detallada es la siguiente:

- El bucle **for** recorre la lista de recetas.
- Para cada objeto **receta** se invoca a su método **obtener_descripcion**.
- El resultado de la invocación se añade a la lista **descripciones**.

Bueno, bonito y barato.

Ejercicio 45 – Programación orientada a objetos: función **filter**

Enunciado:

Sobre una lista de objetos de la clase **Actriz**, obtener aquellas que cumplen con un requisito de altura para protagonizar una película.

Solución:

```
1 class Actriz:
2     def __init__(self, nombre, altura):
3         self.nombre = nombre
4         self.altura = altura
5
6     def mas_165(self):
7         return self.altura > 165
8
9 scarlett = Actriz("Scarlett Johansson", 160)
10 sandra = Actriz('Sandra Bullock', 171)
11 penelope = Actriz('Penelope Cruz', 168)
12
13 actrices = [scarlett, sandra, penelope]
14
15 actrices_mas_165 = filter(Actriz.mas_165, actrices)
16
17 for actriz in list(actrices_mas_165):
18     print(actriz.nombre)
19
```

Salida:

```
Sandra Bullock
Penelope Cruz
```

Explicación:

La función **filter** ya ha sido presentada en sociedad de un ejercicio anterior, pero en este caso le vamos a dar un enfoque orientado a objetos.

Si tenemos que hacer uso de la función **filter** para obtener un subconjunto de una lista de objetos que cumplan una determinada condición, ¿qué mejor manera de hacerlo que teniendo la programación de dicha condición en la propia clase que representa al objeto?

En este ejercicio, la clase **Actriz** dispone de un método llamado **mas_165** si la altura de actriz en cuestión es superior a 165 centímetros.

En la línea 15, se utiliza la función **filter** para obtener de la lista **actrices** otra lista con aquellas que cumplen la condición.

Se podría hacer con funciones en vez de con métodos, pero así queda el código más recogidito y ordenado.

Ejercicio 46 – Programación orientada a objetos: **función map**

Enunciado:

Utilizando la función **map** convertir una relación de objetos de una clase en una tupla de tuplas con los atributos de dichos objetos.

Solución:

```
1 def obtener_tupla(auto):
2     return (auto.marca, auto.modelo)
3
4 class Automovil:
5     def __init__(self, marca, modelo):
6         self.marca = marca
7         self.modelo = modelo
8
9 skoda = Automovil('Skoda', 'Fabia')
10 kia = Automovil('Kia', 'Ceed')
11 peugeot = Automovil('Peugeot', '205')
12
13 autos = [skoda, kia, peugeot]
14
15 map_auto = map(obtener_tupla, autos)
16
17 print(tuple(map_auto))
18
```

Salida:

```
((('Skoda', 'Fabia'), ('Kia', 'Ceed'), ('Peugeot', '205')))
```

Explicación:

En este caso vamos a utilizar la función **map** con una lista de objetos y con una función.

En la línea 15 se invoca a la función **map** pasándole como argumentos una lista de objetos **Automovil** y la referencia a la función **obtener_tupla**. Esta función recibe un objeto **Automovil** y devuelve una tupla con los atributos marca y modelo.

En la variable **map_auto** recogemos todas las conversiones, pero... **map** no devuelve una tupla ni una lista, sino un objeto **iterable**. Por eso en la línea 17 se convierte a tupla mediante la función/constructor **tuple**.

Nota: la función **obtener_tupla** podría ser un método de instancia, como en el ejercicio 45.

Ejercicio 47 – Programación orientada a objetos: **función reduce**

Enunciado:

Obtener el coste total de las suscripciones a las plataformas de televisión a las que estamos suscritos utilizando la función **reduce**.

Solución:

```
1 from functools import reduce
2
3 def acumular_costes(acumulado, suscripcion):
4     return acumulado+suscripcion.coste
5
6 class Suscripcion:
7     def __init__(self, plataforma, coste):
8         self.plataforma = plataforma
9         self.coste = coste
10
11 netflix = Suscripcion('Netflix', 12.99)
12 max = Suscripcion('MAX', 14.99)
13 prime = Suscripcion('Prime Video', 9.99)
14
15 suscripciones = (netflix, max, prime)
16
17 costes = reduce(acumular_costes, suscripciones, 0)
18
19 print(costes) # 37.97
20
```

Explicación:

Como ya vimos en el ejercicio 40, la función **reduce** toma un iterable y genera un único elemento que contiene información obtenida de todos los

elementos de la colección. Es decir, **reduce** a una única “cosa” un conjunto de “cosas”.

Nota: el término “cosa” no es muy técnico, pero a mí me resulta muy claro y espero que también al lector.

En este ejercicio se utiliza la función **reduce** en la línea 17 para acumular los costes de todas las suscripciones a plataformas de películas y series en *streaming*. Para ello se le pasan tres parámetros: la función de cálculo y acumulación de costes (**acumular_costes**), una tupla con las suscripciones (**suscripciones**) y el coste acumulado inicial (**0** en este caso).

La función **acumular_costes**, por su parte, recibe el acumulado parcial y el objeto **suscripción**, sumando al acumulado parcial el coste de la suscripción actual y devolviendo el resultado. La función **map** se encarga de llamar a la función de acumulación tantas veces como objetos hay en la tupla **suscripciones**, consiguiendo el acumulado total.

Ejercicio 48 – Funciones *lambda*

Enunciado:

Crear una función anónima o *lambda* para obtener la relación de clientes con deudas.

Solución:

```
1 class Cliente:
2     def __init__(self, nombre, saldo):
3         self.nombre = nombre
4         self.saldo = saldo
5
6 cliente_1 = Cliente('Ricardo', 300)
7 cliente_2 = Cliente('María', -500)
8 cliente_3 = Cliente('Pep', -250)
9
10 clientes = [cliente_1, cliente_2, cliente_3]
11
12 deudores = filter(lambda cliente: cliente.saldo < 0, clientes)
13
14 for deudor in deudores:
15     print(deudor.nombre, deudor.saldo)
16
```

Salida:

```
María -500
Pep -250
```

Explicación:

Una función *lambda* es una función anónima de un solo uso que se utiliza para compactar el código. Ejecuta una expresión y devuelve el resultado de esta sin necesidad de hacer un **return** explícito.

La sintaxis consiste en escribir la palabra ***lambda*** seguida de una serie de argumentos separados por comas, dos puntos y una expresión cuyo resultado será devuelto automáticamente.

lambda *argumentos* : *expresión*

En este ejercicio, la función ***lambda*** se está utilizando en la línea 12 para indicar a la función ***filter*** la función que debe utilizar para determinar que clientes pasan a formar parte de la lista de deudores. En este ejemplo, aquellos cuyo saldo sea negativo.

Las funciones ***lambda*** se utilizan habitualmente como argumento de las funciones ***map***, ***filter*** y ***reduce***.

Ejercicio 49 – Persistencia de objetos con el módulo **pickle**.

Enunciado:

Almacenar objetos en ficheros utilizando el módulo **pickle**.

Solución:

```
1 import pickle
2
3 class Retroconsola:
4     def __init__(self, nombre, fabricante):
5         self.nombre = nombre
6         self.fabricante = fabricante
7
8 c64_mini = Retroconsola('C64 Mini', 'Retro Games Ltd.')
9
10 # Guardar el objeto en un archivo
11 with open('c64_mini.dat', 'wb') as file:
12     pickle.dump(c64_mini, file)
13
14 # Recuperar el objeto desde el archivo
15 with open('c64_mini.dat', 'rb') as file:
16     c64_mini_recuperada = pickle.load(file)
17
18 #Mostrar el objeto recuperado
19 print(c64_mini_recuperada.nombre, c64_mini_recuperada.fabricante)
20
```

Explicación:

El módulo **pickle** permite almacenar y recuperar objetos en ficheros. Su uso es tremendamente sencillo. Basta con importar el módulo **pickle** (línea 1) realizar operaciones de escritura y lectura.

Las operaciones de escritura se hacen sobre archivos abiertos en modo escritura+binario ('wb', línea 11) mientras que las de lectura sobre archivos abiertos en modo lectura+binario ('rb', línea 15).

La función **dump** (línea 12) permite escribir un objeto (**c64_mini**) en un fichero (**file**). Tan sencillo como eso.

Por su parte, la función **load** (línea 16) permite leer un objeto almacenado en un fichero (**file**) y asignar el resultado a una variable (**c64_mini_recuperada**) teniendo el objeto su tipo original (para ello debe tener visible la declaración de la clase).

El módulo **shelve** dispone de una funcionalidad similar estando además basado en **pickle**. Si tienes interés en serializar y almacenar objetos en fichero, échale un vistazo porque merecerá la pena.

Ejercicio 50 – Introducción a las bases de datos con **SQLite**

Enunciado:

Crear una agenda de contactos que almacene los datos en una base de datos **SQLite**.

Solución:

```
1 import sqlite3
2
3 conn = sqlite3.connect('contactos.db')
4
5 cursor = conn.cursor()
6
7 cursor.execute('''
8     CREATE TABLE IF NOT EXISTS contactos (
9         id INTEGER PRIMARY KEY AUTOINCREMENT,
10        nombre TEXT NOT NULL,
11        email TEXT NOT NULL
12    )
13 ''')
14
15 cursor.execute("INSERT INTO contactos (nombre, email) VALUES (?, ?)",
16                ('Fernando', 'fernando.paniagua.python@gmail.com'))
17
18 conn.commit()
19
20 cursor.execute("SELECT * FROM contactos")
21 contactos = cursor.fetchall()
22
23 for contacto in contactos:
24     print(f"id: {contacto[0]}, nombre: {contacto[1]},
25           email: {contacto[2]}")
26
27 conn.close()
28
```

Salida de la primera ejecución:

Introduce el nombre del contacto: Guido van Rossum Introduce el email del contacto: guido@python.org

```
ID: 1, Nombre: Guido van Rossum, Email: guido@python.org
```

Salida de la segunda ejecución:

```
Introduce el nombre del contacto:Alyssa Coghlan  
Introduce el email del contacto:ncoghlan@gmail.com  
ID: 1, Nombre: Guido van Rossum, Email: guido@python.org  
ID: 2, Nombre: Alyssa Coghlan, Email: ncoghlan@gmail.com
```

Explicación:

Python incluye soporte para SQLite gracias al módulo estándar **sqlite3**.

El funcionamiento es sencillo, pero hay que tener nociones básicas de **bases de datos** y de **SQL**. Si ese es tu caso, no te costará entender las explicaciones. Si no lo es, sería conveniente que te documentases mínimamente sobre estas dos áreas de conocimiento informático.

En la línea 3 se establece la conexión con la base de datos a través de la función **connect**. El argumento es la ubicación en el sistema de archivos y el nombre del fichero que va a contener la base de datos (**contactos.db** en este caso). Se obtiene una conexión con la base de datos.

En la línea 5 se crea un objeto de la clase **Cursor** a través del método **cursor** de la conexión. Este objeto, llamado **cursor** en el ejercicio, nos dará la posibilidad de realizar operaciones sobre la base de datos.

Entre las líneas 7 y 13 se crea la tabla **contactos**, ejecutando una sentencia **SQL** a través del método **execute** del cursor.

En las líneas 15 y 16 se inserta un registro en la base de datos.

En la línea 18 se indica a la conexión que haga firmes los cambios a través del método **commit**.

En la línea 20 se ejecuta una sentencia SQL de consulta.

En la línea 21 se está obteniendo el resultado de la consulta a través del método **fetchall** del cursor. Este método devuelve todos los registros de una única vez en una lista que contiene una tupla por cada registro.

Entre las líneas 23 y 25 se recorre la lista y se muestran los datos.

En la línea 27 se está realizando el cierre de la conexión de base de datos mediante el método **close** de la conexión.



Este ejercicio es muy sencillo y no pretende más que dar a conocer este interesante y útil módulo. Para sacarle provecho hay que tener conocimientos sobre bases de datos y SQL. Si no los tienes, lector, dedícale un buen rato de tu tiempo de formación a ello cuando sea posible. Las bases de datos son una potente herramienta dentro del mundo de la programación.

Ejercicio 51 – ¿Tienes hora?

Enunciado:

Obtener la hora del sistema utilizando el módulo **datetime**.

Solución:

```
1 import datetime
2
3 hora_actual = datetime.datetime.now().strftime("%H:%M:%S")
4
5 print("Hora actual:", hora_actual) # Hora actual: 10:19:01
6
```

Explicación:

Como no podría ser de otra forma, Python dispone de módulos para manejar fechas y horas, siendo **datetime** el más usado.

En este ejercicio se está utilizando para obtener la hora del sistema. Analicemos la solución.

En la línea 3 se está accediendo al método de clase **now** que se encuentra dentro de la clase **datetime** que a su vez se encuentra en el módulo con el mismo nombre. La secuencia **datetime.datetime.now()** es equivalente a **módulo.clase.método()**.

Esta primera parte de la sentencia devuelve un objeto **datetime**, con información sobre la fecha y hora del sistema. La segunda parte, correspondiente al método **strftime** se utiliza para dar formato a la información. En este caso se le está indicando que se quiere mostrar una cadena con la hora (%H), dos puntos, el minuto (%M), dos puntos y el segundo (%S).

Ejercicio 52 – ¿Cuántos años tengo?

Enunciado:

Calcular la edad actual a partir de la fecha de nacimiento.

Solución:

```
1 import datetime
2
3 fecha_nacimiento = input('Fecha de nacimiento (DD-MM-AAAA):')
4 componentes_fecha = fecha_nacimiento.split('-')
5 fecha_nacimiento = datetime.datetime(
6     int(componentes_fecha[2]),
7     int(componentes_fecha[1]),
8     int(componentes_fecha[0]))
9
10 fecha_actual = datetime.datetime.now()
11
12 edad = fecha_actual.year - fecha_nacimiento.year
13
14 if (fecha_actual.month, fecha_actual.day) < \
15     (fecha_nacimiento.month, fecha_nacimiento.day):
16     edad -= 1
17
18 print(f'Tienes {edad} años')
19
```

Salida (ejecución el día 23/02/2025):

```
Introduce tu fecha de nacimiento (DD-MM-AAAA):21-02-1941
Tienes 84 años
```

Explicación:

En este ejercicio se hace un uso del módulo **datetime** para obtener una diferencia de fechas.

En la línea 1 se solicita la fecha de nacimiento en formato **DD-MM-AAAA**.

En la línea 4 se utiliza el método **split** de **str** para obtener una lista con las partes de la fecha escrita por el usuario, utilizando el guion como separador.

Entre las líneas 5 y 8 se está construyendo un objeto de la clase **datetime** con los días, meses y años almacenados en la lista obtenida con **Split**.

La línea 10 obtiene la fecha y hora del sistema.

En la línea 12 se resta el año de nacimiento del año actual, obteniendo su edad, pendiente de saber si ha cumplido años o no. El año en un objeto **datetime** se encuentra en el atributo **year**.

Entre las líneas 14 y 16 se está comprobando si el mes y el día actual es menor que el mes de nacimiento. Si es así quiere decir que el usuario no ha cumplido años en el año presente por lo que hay que restar 1 a su edad.

En Python, cuando se comparan dos secuencias (en este caso dos tuplas) se van comparando los elementos secuencialmente. En primer lugar, se comparan los primeros elementos de cada una de las secuencias. Si son distintos, se resuelve la comparación con el resultado. Si son iguales, se pasan a comprobar los dos segundos y así sucesivamente.

Ejercicio 53 – Propósito

Enunciado:

Me he propuesto no tomar arroz con leche en los próximos 100 días. ¿Hasta cuándo voy a tener que estar privándome de tan maravilloso manjar?

Solución:

```
1 from datetime import datetime, timedelta
2
3 fecha_actual = datetime.now()
4
5 incremento = timedelta(days=100)
6
7 fecha_final = fecha_actual + incremento
8
9 print(fecha_final.strftime('%d-%m-%Y'))
10 # Salida: 03-06-2025
11 # Fecha de ejecución: 03-02-2025
12
```

Explicación:

Otro ejercicio más de fechas y horas. En este caso para saber qué fecha será dentro de un número determinado de días.

Para conseguir obtener una fecha como consecuencia de un incremento o decremento sobre otra se utiliza una clase llamada **datetime**.

En la línea 5 se crea un **datetime** que almacena un desplazamiento de 100 días. Este desplazamiento se suma a la fecha de hoy, dando como resultado la fecha que será dentro de este plazo, almacenada en la variable **fecha_final**.

En la línea 9 se está dando formato a la salida para que sea más legible.

Ejercicio 54 – Manteniendo el tipo

Enunciado:

Examinar y convertir los tipos de las variables

Solución:

```
1 nombre = 'María'
2 edad = 23
3 altura = 1.71
4 programador = True
5 impedancia = 10 + -286.894j
6 impares = [1, 3, 5, 7]
7
8 print(type(nombre)) # <class 'str'>
9 print(type(edad)) # <class 'int'>
10 print(type(altura)) # <class 'float'>
11 print(type(programador)) # <class 'bool'>
12 print(type(impedancia)) # <class 'complex'>
13 print(type(impares)) # <class 'list'>
14
15 edad = str(edad)
16 print(type(edad)) # <class 'str'>
17
18 impares = tuple(impares)
19 print(type(impares)) # <class 'tuple'>
20
```

Explicación:

Todas las variables en Python tienen un tipo. En las primeras 6 líneas se están declarando varias variables a las que se les asignan distintos tipos de datos. Compara la asignación con el resultado de mostrar el tipo (con la función **type**) de cada una de las variables, entre las líneas 8 y 13. En función del valor asignado Python asigna el tipo. Este tipado se llama **dinámico con inferencia de tipo**. Es decir, que el tipo de la variable puede cambiar durante

la ejecución del programa y que dicho tipo se asignará infiriéndose en función del valor asignado.

Puede que queramos que una variable cambie de tipo, pero no queremos asignarle un nuevo valor. Esto se hace con las funciones/constructores de los tipos, como **str**, **int**, **bool**, **float**, **bin**, **tuple**, **list**, **set** o **dict**. Si a estas funciones se le pasa como argumento una variable **compatible** devolverá una nueva variable con el tipo indicado por la función.

En este ejemplo se utilizan las funciones **str** y **tuple** en las líneas 15 y 18, para convertir una variable **int** a **str** y una variable **list** a **tuple** respectivamente.

Ejercicio 55 – Excepciones

Enunciado:

Capturar y tratar la excepción provocada por una división entre cero.

Solución:

```
1 try:
2     resultado = 100 / 0
3     print('Resultado:', resultado)
4 except ZeroDivisionError as zde:
5     print('Ha ocurrido un error:',
6     zde)
```

Salida:

```
Ha ocurrido un error: division by zero
```

Explicación:

El manejo de excepciones daría para un libro aparte, pero al menos debemos conocerlas mínimamente.

Cuando una sentencia puede producir un error (llamado excepción) este puede tratarse de la siguiente manera.

La línea que potencialmente puede generar el error se mete dentro de un bloque de código identificado por la palabra **try** y el código que se ejecutará si dicho error se produce en otro bloque de código identificado con la palabra **except**.

En este ejercicio se intenta ejecutar una división de un número entero entre 0 en la línea 2. Como sabemos que esa operación pueda provocar un error, metemos la sentencia en el bloque **try** declarado en la línea 1.

Si se produce una excepción de tipo **ZeroDivisionError** (una de las excepciones contempladas por defecto en Python) esta se “captura” en la línea 4 y se almacena la información del error en una variable que se ha llamado **zde**. En la línea 5 se realiza la acción asociada a la detección del error, en este caso mostrar un mensaje.

¿Y qué pinta la línea 3 en esto? Es muy sencillo. La línea 3 solo se ejecuta si la línea 2 no ha producido error. Esa línea está condicionada a las líneas anteriores del bloque **try**. Todo lo que está en el bloque **try** se ejecutará de manera secuencial hasta que se produzca un error, entonces se abortará la ejecución y se pasará al tratamiento de dicho error.

¿Y qué pasa si ocurre un error que no está contemplado en el bloque **except**? En realidad, se pueden indicar varios bloques **except** para capturar los diferentes errores, pero si ocurre alguno que no está capturado el programa dará un error. Tan sencillo como eso.

Ejercicio 56 – Comprobando tipos

Enunciado:

Crear una función que sume dos números enteros después de comprobar que los tipos de los argumentos son correctos.

Solución:

```
1 def sumar(s1, s2):
2     if not isinstance(s1, int) or not isinstance(s2, int):
3         raise TypeError('Los argumentos deben ser enteros')
4     else:
5         return s1 + s2
6
7 try:
8     resultado = sumar(8.5, 7)
9     print(f"El resultado es: {resultado}")
10 except TypeError as te:
11     print(te)
12
```

Salida:

```
Los argumentos deben ser enteros
```

Explicación:

Tener un lenguaje de tipado dinámico es útil para unas cosas y una preocupación para otras. Aquellas personas lectoras de este libro que vengáis de programar en lenguajes de tipado estático os haréis algunas preguntas. ¿Qué ocurre si paso un argumento a una función que no es del tipo adecuado? ¿El programa producirá un error de ejecución? ¿Hará cosas raras? ¿La pizza con piña o sin piña?

Como casi siempre la respuesta es que depende de varios factores y de lo que la función haga. A veces fallará, a veces hará mal su trabajo y a veces

hará algo con sentido, aunque esto es menos probable. De lo de la piña en la pizza no voy a opinar que hay personas para las que este asunto es muy serio.

Sea como sea, si quieres verificar los tipos de las variables que entran como argumentos en una función lo puedes hacer.

En la línea 2 se está comprobando si los argumento **s1** o **s2 no son de tipo int**, utilizando la función **isinstance**. Esta función comprueba el objeto pasado como primer argumento es del tipo indicado en el segundo. Si alguno de los argumentos no es **int** crea una nueva excepción y la lanza al sistema. En este caso está construyendo una excepción de tipo **TypeError** (es la que se utiliza cuando los tipos no son adecuados) y lanzándola con la instrucción **raise**.

Esta operativa es parecida a la que hace Python cuando intentamos dividir un número entre 0. Comprueba el valor y “lanza” una excepción. El que estaba intentando hacer la operación debe decidir si la detecta y trata o deja que el programa falle.

En este caso, entre las líneas 7 y 10 se muestran la estructura de código **try-except** que se encarga de detectar que se ha producido este tipo de error y de hacer un tratamiento adecuado de este.

Ejercicio 57 – Aserciones. ¿Eso qué es?

Enunciado:

Introducir mecanismos para detectar que un bloque de código funciona correctamente.

Solución:

```
1 def obtener_mayor(lista):
2     mayor = lista[0]
3     for numero in lista:
4         if numero < mayor:
5             mayor = numero
6     return mayor
7
8 assert obtener_mayor([-4, 10, 2])==10
9
10 numeros = [3, 100, -4, 14]
11 print(obtener_mayor(numeros))
12
```

Explicación:

Una aserción en Python es un mecanismo de depuración que se utiliza para verificar que se cumpla una condición durante la ejecución del programa.

En la línea 8 se está creando una aserción con la palabra reservada **assert** en que se está indicando que la llamada a **obtener_mayor** con esa lista de números (-4, 10 y 2) debe devolver un 10. Si no fuese así, Python lanzará una excepción del tipo **AssertionError**.

En este ejemplo hay un error en la línea 4. El operador de comparación debería ser >. La aserción prueba la función y genera una **AssertionError** para avisar del mal funcionamiento de dicha función. Prueba el código tal cual está escrito y después cambia el operador. Observa la diferencia.

Ejercicio 58 – Almacenamiento de funciones en variables

Enunciado:

Almacenar funciones una lista y ejecutarlas en secuencia.

Solución:

```
1 def ejecutador(tarea):  
2     tarea()  
3  
4 def hacer_tarea_1():  
5     print('Haciendo la tarea 1')  
6  
7 def hacer_tarea_2():  
8     print('Haciendo la tarea 2')  
9  
10 tareas = [hacer_tarea_1, hacer_tarea_2]  
11  
12 for tarea in tareas:  
13     ejecutador(tarea)  
14
```

Salida:

```
Haciendo la tarea 1  
Haciendo la tarea 2
```

Explicación:

Python es un lenguaje multiparadigma y uno de los que contempla es la programación funcional. En este paradigma, las funciones tienen un trato

especial. Una de las capacidades que posee es que se pueden asignar a variables.

En este ejercicio se están creando 3 funciones. La primera de ellas, llamada **ejecutador** recibe un argumento que se ha llamado **tarea** y que será una función. En la línea 2 se ejecutará la tarea recibida.

Las otras dos funciones muestran un mensaje por pantalla.

Lo interesante viene a continuación. En la línea 10 se crea una lista con los nombres de las tareas (fíjate bien: ni llevan comillas ni paréntesis al final).

En las líneas 12 y 13 se recorre la lista y se van pasando las funciones a la función **ejecutador** que procede a ejecutarlas.

Hemos creado un motor de ejecución. Es muy sencillo pero da pistas de qué cosas se pueden hacer con Python.

Ejercicio 59 – Estructuras de datos: creando una cola

Enunciado:

Crear una cola utilizando orientación a objetos.

Solución:

```
1 class Cola:
2     def __init__(self):
3         self.items = []
4
5     def agregar(self, item):
6         self.items.append(item)
7
8     def extraer(self):
9         if self.esta_vacia():
10             raise Exception('No hay elementos')
11         return self.items.pop(0)
12
13     def esta_vacia(self):
14         return len(self.items) == 0
15
16     def mostrar_contenido(self):
17         print(self.items)
18
19 tareas = Cola()
20 tareas.agregar('Tarea 1')
21 tareas.agregar('Tarea 2')
22 tareas.agregar('Tarea 3')
23 tareas.mostrar_contenido()
24 print('Siguiete tarea:',tareas.extraer())
25 tareas.mostrar_contenido()
26
```

Salida:

```
['Tarea 1', 'Tarea 2', 'Tarea 3']  
Siguiente tarea: Tarea 1  
['Tarea 2', 'Tarea 3']
```

Explicación:

Una cola es una estructura de datos en la que se agregan y extraen elementos según la regla “el primero que entra en la cola es el primero que sale de ella” (como en la cola del supermercado). También se conoce a esta estructura por las siglas **FIFO (First In, First Out)** que viene a ser lo mismo, pero en inglés.

Aunque Python proporciona una cola a través de la clase **deque** del módulo **collections** de la librería estándar, vamos a implementar una cola propia muy básica utilizando orientación a objetos y listas.

En la solución se define una clase **Cola** en cuyo constructor se crea un atributo con el nombre **ítems** que contiene una lista vacía.

Los métodos relacionados con la gestión de la cola son los siguientes:

- **agregar**: recibe un elemento (**ítem**) y lo añade al final de la lista con el método **append**.
- **extraer**: devuelve el elemento de la lista que se encuentra en la posición 0 y lo elimina mediante el método **pop**. Extraer utiliza el método **esta_vacia** para comprobar el estado de la lista. Si se intenta extraer un elemento cuando la lista no tiene ninguno generar una excepción
- **esta_vacia**: devuelve **True** si la longitud de la lista es 0.

Según estos métodos, cuando se agrega un elemento a la lista este se pone en el último lugar, quedando siempre en primer lugar el primero que se agregó. Cuando se extraen, siempre se obtiene el primero, por lo que la forma **FIFO** de funcionar de esta estructura de datos es la correcta.

Ejercicio 60 – Búsqueda de palabras

Enunciado:

Buscar el nombre de filósofo más parecido al nombre introducido por el usuario.

Solución:

```
1 # pip install levenshtein
2 from Levenshtein import distance
3 import math
4
5 filosofos = ['Sócrates', 'Schopenhauer', 'Nietzsche']
6
7 distancia = math.inf
8
9 nombre = input('Nombre del filósofo:')
10 for filosofo in filosofos:
11     if distance(nombre, filosofo) < distancia:
12         distancia = distance(nombre, filosofo)
13         nombre_candidato = filosofo
14
15 print(f'El filósofo más cercano a {nombre} es {nombre_candidato}')
```

Salida:

```
Nombre del filósofo:chopenjauer
El filósofo más cercano a chopenjauer es Schopenhauer
```

Explicación:

Este ejercicio sirve para presentar el instalador de paquetes de Python que no forman parte de la librería estándar de Python. Se llama **pip** que es el acrónimo recursivo de “*Pip Installs Packages*” y es realmente sencillo de utilizar.

En este ejercicio se utiliza para instalar un paquete llamado **levenshtein** que permite, entre otras cosas, calcular la “distancia” entre dos palabras. Es

decir, cuánto hay que cambiar en una palabra para que sea otra o lo que es lo mismo, saber cuánto se parecen dos palabras entre sí.

Para instalar este paquete basta con escribir en una terminal del sistema operativo que estés usando la secuencia **pip install levenshtein** y aceptar pulsando la tecla "Enter". **Pip** se encarga de descargar el módulo e instalarlo en tu ordenador. Este módulo y otros cuantos cientos de miles más se encuentran almacenados en **pypi.org**, que es el repositorio de módulos de Python. Si quieres hacer algo grande en Python es un buen lugar para buscar módulos que te ayuden en tu proyecto.

En cuanto a la solución del ejercicio la explicación es la siguiente.

Se establece una distancia inicial infinita mediante la constante **inf** del módulo **math** y se almacena en la variable **distancia** (línea 7). Después se introduce el nombre de un filósofo y se recorren todos los filósofos de la lista. Si la distancia calculada entre el nombre introducido y el nombre del filósofo que se está evaluando en esa iteración del bucle es menor que la almacenada en la variable **distancia** ese filósofo será el mejor candidato hasta el momento, actualizándose el valor de **distancia**. La distancia se calcula mediante la función **distance** del módulo **levenshtein**.

Al finalizar la ejecución, la variable **nombre_candidato** tendrá el nombre del filósofo más parecido al introducido por el usuario.

Este ejercicio con Sócrates no parece muy necesario, pero con "Chopenjaeur" seguro que resulta útil.

Ejercicio 61 – Parando el tiempo

Enunciado:

Simular que el programa está dedicando un tiempo a pensar deteniendo la ejecución de este mediante la función **sleep** del módulo **time**.

Solución:

```
1 import time
2
3 input('Piensa un número y pulsa la tecla Enter:')
4 print('Pensando:',end='')
5 for i in range(5):
6     print('.',end='')
7     time.sleep(1)
8 print()
9 print('El número que has pensado es 3 ¿verdad?')
10
```

Salida:

```
Piensa un número y pulsa la tecla Enter
Pensando:.....
El número que has pensado es 3 ¿verdad?
```

Explicación:

El módulo **time** tiene, entre otras cosas, una función que se llama **sleep** que suspende la ejecución de un programa durante un número determinado de segundos.

En este ejercicio, el bucle repite 5 veces la ejecución de las líneas 6 y 7. En la línea 6 se detiene la ejecución del programa durante 1 segundo con la función **sleep**. Esto provocará que los puntos que muestra la línea 7 vayan apareciendo poco a poco. Así parece que está pensando.

Ejercicio 62 – Organización de proyectos: módulos

Enunciado:

Crear un módulo que contenga funciones para el cálculo del área. Utilizar la relativa al cálculo del área del círculo en otro programa Python.

Solución:

Código de **ejercicio_62_funciones.py**

```
1 import math
2
3 def calcular_area_rectangulo(base, altura):
4     return base * altura
5
6 def calcular_area_circulo(radio):
7     return math.pi * radio ** 2
8
```

Código de **ejercicio_62_cliente.py**

```
1 from ejercicio_62_funciones import calcular_area_circulo
2
3 if __name__ == '__main__':
4     radio = 5
5     area = calcular_area_circulo(radio)
6     print(f'El área del círculo es: {round(area,2)}')
7
```

Salida:

El área del círculo es: 78.54

Explicación:

Cuando un programa Python necesita utilizar algo que se encuentra en otro programa (un módulo), lo que hacemos es importar o bien todo el módulo o bien solo la parte que vamos a utilizar.

En este ejercicio participan dos programas: **ejercicio_62_funciones.py** y **ejercicio_62_cliente.py**. Lo novedoso está en el segundo programa.

La primera línea de **ejercicio_62_cliente.py** está importando del módulo **ejercicio_62_funciones** una entidad llamada **calcular_area_circulo**, que en este caso es una función.

Otra novedad es la sentencia que se encuentra en la línea 3 de **ejercicio_62_cliente.py**. La evaluación **if __name__ == '__main__'** está comprobando si el nombre del módulo que está ejecutando esa línea se llama **'__main__'**. Este nombre es especial, ya que lo tiene el módulo que se ejecuta en primer lugar como script principal. Cuando el módulo es importado desde otro, la variable **__name__** tiene el nombre del módulo en el que se encuentra.

Todo este galimatías significa que el contenido de ese bloque condicional solo se ejecuta si se ha ejecutado directamente el módulo que lo contiene, no ejecutándose si dicho módulo ha sido importado.

Ejercicio 63 – *args

Enunciado:

¿Cómo conseguir que una función reciba un número variable de argumentos?

Solución:

```
1 def mostrar(*args):
2     for arg in args:
3         print(arg)
4
5 mostrar('Hola')
6 mostrar('Hola', 'Mundo')
7 mostrar('Hola', 'Mundo', 'Python')
8
```

Salida:

```
Hola
Hola
Mundo
Hola
Mundo
Python
```

Explicación:

Los argumentos en Python tienen algunas notaciones curiosas. Una de ellas es la consistente en anteponer un asterisco al nombre del argumento, que se le suele llamar **args**, aunque este nombre no es obligatorio.

Esto significa que ese argumento puede recibir un número indeterminado de argumentos posicionales en la llamada a la función que se recogerán en una tupla. En la llamada a la función se separan por comas.

Por ejemplo, la función **print** recibe como primer argumento ***values**

Ejercicio 64 – **kwargs

Enunciado:

Crear una función que obtenga los nombres de los argumentos recibidos cuando estos no son conocidos en el momento de la programación.

Solución:

```
1 def obtener_nombres(**kwargs):
2     nombres = []
3     for nombre in kwargs.keys():
4         nombres.append(nombre)
5     return nombres
6
7 nombres = obtener_nombres(cpu='intel', ram='32gb')
8 print(nombres) # ['cpu', 'ram']
9
```

Explicación:

En esta notación el nombre del argumento se precede de dos asteriscos. En este caso, el argumento será un diccionario y la llamada a la función consistirá en un número indeterminado de argumentos con nombre (*keyword arguments*), como se puede ver en la línea 7.

Los nombres **args** y **kwargs** son convenciones, pero no obligaciones sintácticas. Estos nombres siguen las reglas de los nombres de variable de Python.

El uso de ***args** y ****kwargs** excede el objetivo de este libro, pero puedes profundizar fácilmente en su conocimiento si te interesa buscando ejemplos en la web.

Ejercicio 65 – Formateando cadenas con el método **format**

Enunciado:

Mostrar un texto por pantalla dándole formato utilizando el método **format** de la clase **str**.

Solución:

```
1 hijo = 'Luke Skywalker'
2 padre = 'Darth Vader'
3 porcentaje_adn = 0.413284
4
5 print("El hijo es: {}, el padre es: {}, y el porcentaje de ADN es: {:.2f}%".
6       format(hijo, padre, porcentaje_adn * 100))
7
```

Salida:

```
El hijo es: Luke Skywalker, el padre es: Darth Vader, y
el porcentaje de ADN es: 41.33%
```

Explicación:

El método **format** permite dar formato de manera “fácil” a una cadena de caracteres. La explicación de las líneas 5 y 6, donde se hace uso de este método de la clase **str**, es la siguiente.

La primera parte del argumento que se pasa a la función **print** es una cadena que contiene tres pares de llaves de apertura y cierre. Las dos primeras están vacías mientras que en la tercera se encuentra la secuencia de caracteres “**:.2f**”.

Sobre dicha cadena se invoca al método **format** al que se le pasa como argumentos las variables **hijo**, **padre** y la expresión **porcentaje_adn * 100**.

La función **format** sustituye las llaves por los valores de las variables en el orden en que unas y otras aparecen: el primer par de llaves se asocia con la variable **hijo**, el segundo con **padre** y el tercero con la expresión **porcentaje_adn * 100**. En este último caso se indica, además que se quiere mostrar el valor de la expresión con un formato concreto.

La secuencia de caracteres **":.2f"** significa que se inicia el formato **":"**, se quieren mostrar dos decimales **".2"** y que se desea expresar como un número decimal **"f"**.

Como se puede suponer, el método **format** tiene muchísimas más posibilidades. Este ejercicio solo es una presentación.

Ejercicio 66 – Trabajando en Unix/Linux/macOS: **shebang**

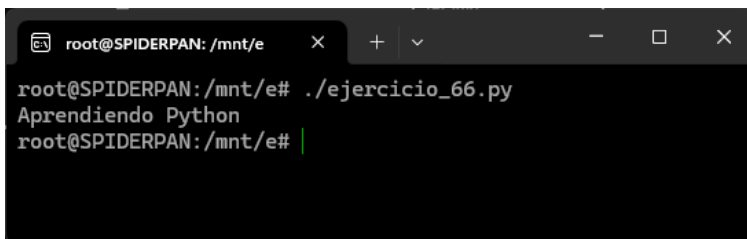
Enunciado:

Indicar a Ubuntu qué intérprete de Python debe utilizar para ejecutar un script.

Solución:

```
1 #!/usr/bin/python3
2
3 print('Aprendiendo Python')
4
```

Salida:



```
root@SPIDERPAN: /mnt/e  x  +  v  -  □  x
root@SPIDERPAN: /mnt/e# ./ejercicio_66.py
Aprendiendo Python
root@SPIDERPAN: /mnt/e#
```

Explicación:

Si estás trabajando en algún sistema operativo derivado de Unix, ya sea un descendiente directo como macOS o un sistema inspirado en él como Linux, esto te interesa.

Los scripts de Python deben llevar una primera línea conocida, entre otros nombres, como **shebang** en la que se indica, precedida por un asterisco y un símbolo de admiración, la ruta completa y el nombre del intérprete de Python que se quiere utilizar. Esto permite ejecutar el script simplemente escribiendo su nombre en una consola.

Ejercicio 67 – Procesando textos: El Quijote

Enunciado:

Contar el número de palabras distintas que tiene El Quijote y averiguar si contiene o no una palabra introducida por el usuario.

Solución:

```
1 import re
2
3 simbolos = r'[.,;:¿?¡!(«»"\'--] '
4
5 with open('elquijote.txt', 'r', encoding='utf-8') as f:
6     texto = f.read()
7
8 texto_limpio = re.sub(simbolos, '', texto)
9
10 palabras_distintas = set(texto_limpio.upper().split())
11 print(len(palabras_distintas))
12
13 palabra = input('Introduce una palabra:')
14 if palabra.upper() in palabras_distintas:
15     print(f'La palabra {palabra} está en el texto')
16 else:
17     print(f'La palabra {palabra} no está en el texto')
18
```

Salida:

```
22959
Introduce una palabra:mameluco
La palabra mameluco está en el texto
```

Explicación:

Este ejercicio hace un proceso con un texto almacenado en un fichero. En este caso utilizaremos la célebre obra de Miguel de Cervantes.

En la primera línea se puede observar la importación de un módulo nuevo que no había aparecido hasta ahora: el módulo **re**. Este módulo trae herramientas para trabajar con expresiones regulares (*regular expressions* en inglés, de ahí su nombre).

En la línea 3 se crea una cadena de caracteres llamada **símbolos**, pero se hace de una manera especial. Se antepone el carácter **r**. Esto indica que la cadena es de tipo **raw** o cruda, con lo que se indica a Python que no tiene que procesar lo que hay dentro, evitando a sí la necesidad de “escapar” caracteres que tengan algún tipo de significado, como el carácter ****. Esto es útil cuando se trabaja con expresiones regulares.

Las líneas 5 y 6 simplemente leen todo el texto de El Quijote almacenado en un fichero y se guarda en la variable **texto**. Esto puede parecer mucho, pero Python puede con casi todo.

En la línea 8 empieza la magia. Se invoca a la función **sub** del módulo **re** y se le pasa la cadena **símbolos**, una cadena vacía y la variable **texto**. Esta función sustituirá cualquiera de los caracteres contenidos en la cadena **símbolos** que se encuentren en la cadena almacenada en la variable **texto** por una cadena vacía. De esta manera eliminamos caracteres “conflictivos” del texto original, como comas, puntos y otros signos de puntuación. El texto resultante se almacena en la variable **texto_limpio**.

En la línea 10, el contenido de **texto_limpio** se convierte a mayúsculas y se separa en palabras mediante el método **split**. El resultado se pasa como argumento a la función/constructor de conjuntos **set** que eliminará los duplicados.

En este punto tenemos en la variable de tipo conjunto **palabras_distintas** con cada una de las palabras distintas que contiene la obra (sin duplicados, que no se nos olvide).

Aprende un poco de Python

La línea 11 muestra el tamaño de ese conjunto, que será el número de palabras diferentes que tiene El Quijote. No será exacto, pero se parecerá mucho al real.

A continuación, se solicita una palabra al usuario (línea 13) y se pregunta si dicha palabra convertida a mayúsculas se encuentra en el conjunto.

Recuerda que en la línea 10 se convirtió todo el texto a mayúsculas. Esto se hace para evitar que la misma palabra escrita en mayúsculas o minúsculas sea tratada como distinta por el programa.



Puede parecer mucho proceso, pero se encuentra contenido en solo 17 líneas de código. No está nada mal, ¿verdad?

Ejercicio 68 – Obteniendo datos de la web con **request**

Enunciado:

Descargar el código de la página **html** referente al videojuego **Monkey Island** (otro de mis favoritos) de **Wikipedia** utilizando la librería **request** y almacena el contenido en un fichero.

Solución:

```
1 # pip install requests
2
3 import requests
4
5 url = 'https://es.wikipedia.org/wiki/Monkey_Island'
6
7 response = requests.get(url)
8
9 if response.status_code == 200:
10     with open('monkey_island.html', 'w', encoding='utf-8') as fichero:
11         fichero.write(response.text)
12     print("Página descargada con éxito.")
13 else:
14     print(f"Error al descargar la página. Código de estado: {response.status_code}")
15
```

Salida:

```
Página descargada con éxito.
```

Además, se genera el fichero 'monkey_island.html' con el código html de la página de Wikipedia.

Explicación:

El último ejercicio, pero no el menos importante. Ni mucho menos.

El módulo **requests** permite realizar peticiones HTTP para acceder a recursos publicados en la web, como páginas o servicios web. Es bastante sencillo de utilizar si lo que se quiere es realizar peticiones simples, como en este ejercicio.

Este módulo no forma parte de las librerías estándar de Python, por lo que hay que instalarlo utilizando **pip**.

En la línea 5 se define una variable con una URL que apunta a una página de Wikipedia.

En la línea 5 se hace una petición de tipo **GET** mediante la función **get** del módulo **requests**, que recibe como argumento la URL anterior. Esta llamada devuelve un objeto con la respuesta de la petición, que en este caso se llama **response**.

La línea 9 accede al código de estado de la petición y se comprueba si es **200**. Este código se encuentra en el atributo **status_code** de la respuesta. En HTTP, el código 200 significa que todo ha ido bien. Es un **OK**. En este ejercicio, si la comunicación se ha desarrollado satisfactoriamente se escribe el texto que nos ha devuelto el servidor (el código de la página HTML) en un fichero. Dicho texto se encuentra en el atributo **text** de la respuesta.

Esto es todo. Nada más. Nada menos.

Las llamadas HTTP tienen muchísima más casuística, pero como en otras ocasiones, esto solo es una presentación de la funcionalidad proporcionada por Python, en este caso a través del módulo **requests**.

CONCLUSIONES

Aprender a programar requiere tiempo y esfuerzo. Hay personas que aprenden más rápido y otras más despacio, pero mi experiencia como formador me llega a concluir que prácticamente todo el mundo aprende a programar. Sea cual sea tu caso, si quieres ser programador o programadora de software no dudes que es posible.

En este breve libro he presentado algunas características básicas de Python, haciendo un somero repaso de varias de sus capacidades. Ten en cuenta que esto es solo el principio. Hay muchísimo más. El potencial de este lenguaje es prácticamente infinito.

Me despido hasta el momento en el que tengamos la oportunidad de coincidir, ya sea en el aula o en otro libro. Espero que sea pronto. También espero que el libro te resulte interesante y, sobre todo, útil.

Un fuerte abrazo.

Fernando Paniagua.